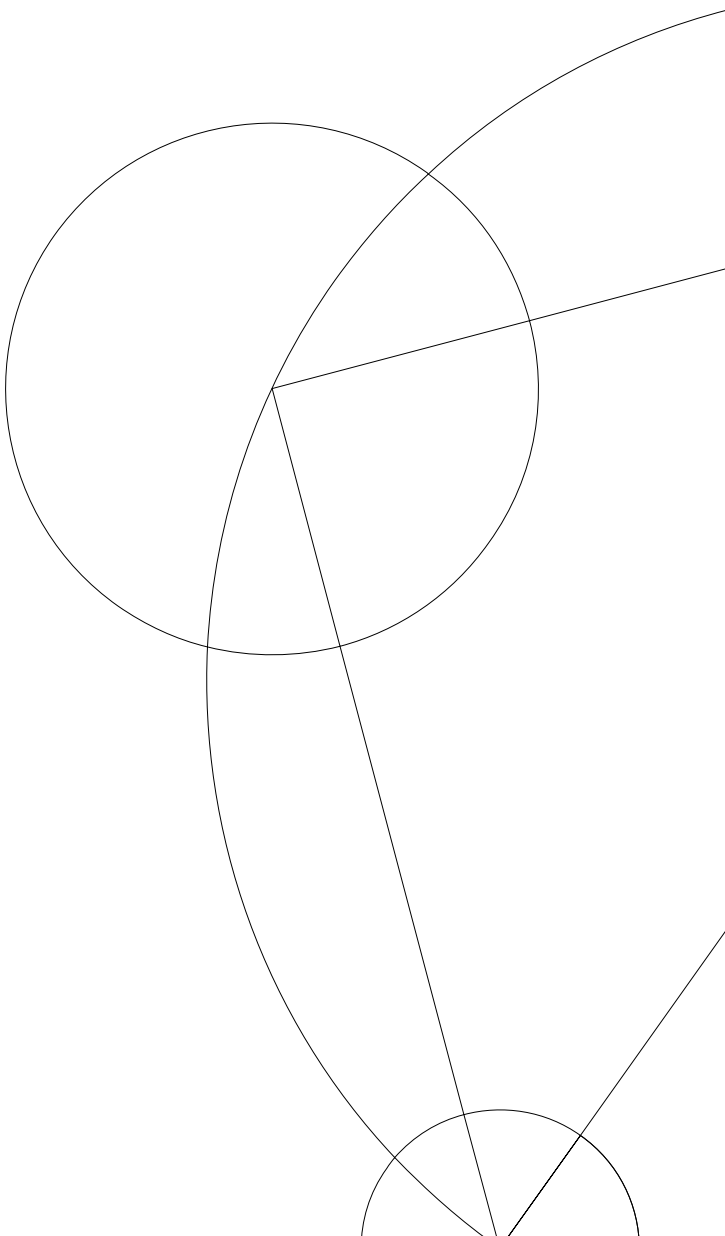
UNIVERSITY OF COPENHAGEN
FACULTY OF SCIENCE

BSc Thesis

Oliver Thejl Eriksen and Daniel de Oliveira Stephensen

Planar Convex Hulls



Counselor: Mikkel Abrahamsen

Department of Computer Science. June 13, 2022

Abstract

This thesis thoroughly investigates the theoretical foundation of the planar convex hull problem. In its entireness, the thesis can be considered an encapsulation of the most important aspects of the problem.

The strongest lower bound for time complexity is proven, some of which is a modern and more easily digestible reiteration of previous work, along with new lemmas that previously were omitted. A progression of algorithms are presented along with proofs of time complexity and correctness. A probabilistic variant is analyzed, outlining circumstances for which the convex hull can be computed in expected linear time and proving one such. Finally the described algorithms are implemented and the number of comparisons are compared for a number of distributions.

This work is part survey, part elaboration on meager proofs. Original contributions include: Additions to a proof of the lower bound of the multiset support size verification problem, algorithm for binary searching convex hulls, explanation of algebraic decision trees and reduction proofs, intuition behind properties of the orientation test, a generalization of the Akl-Toussaint heuristic, implementation and analysis of comparisons in practice, curation of material on the convex hull problem along with an assortment of original figures.

Contents

1 Terminology

Throughout this paper various abbreviations and definitions will be used. Although we define these terms as we use them and take great care not to dilute our results with cluttered notation, we still provide the following reference list to keep handy when reading the paper.

Q	Multiset of points used as input to the convex hull problem.
$\mathrm{CHV}(Q)$	Set of vertices of the convex hull of Q.
$n = \lvert Q \rvert$	Number of input points.
$h = \lvert \mathrm{CHV}(Q) \rvert$	Number of extremal points on the convex hull.
$T(\textsc{Algorithm})$	Time complexity of $\textsc{Algorithm}$
$P_1 \succeq P_2$	Problem P_2 reduces to P_1. Informally, P_2 is easier than P_1
$[x_1, \ldots, x_k]$	Multiset of k elements.
$\langle x_1, \ldots, x_k \rangle$	Cyclic sequence of k elements.

2 Introduction

Given a finite multiset of points Q in the plane, the convex hull of Q is the smallest convex set containing Q or equivalently the set of convex combinations of Q. Graphically this can be interpreted as a convex polygon containing Q with vertices in Q. The convex hull is canonically represented by its extremal points, corresponding to vertices of the polygon. This representation, denoted $\mathrm{CHV}(Q)$, is what we concern ourselves with.

The convex hull is a fundamental structure in computational geometry, used for collision detection [12], path finding [13], image processing [10], among others. The problem has several algorithmic variants, some of which we order here from hardest to easiest[1]:

- **Convex hull sequence problem**: Compute $\mathrm{CHV}(Q)$ as a cyclically ordered sequence starting from a predefined extremal point (e.g. leftmost point).
- **Convex hull set problem**: Compute $\mathrm{CHV}(Q)$ as an unordered set.
- **Convex hull cardinality problem**: Compute $|\mathrm{CHV}(Q)|$.
- **Convex hull cardinality verification problem**: Given an integer k, determine if $k \stackrel{?}{=} |\mathrm{CHV}(Q)|$.

Just as the output of the problem can be specified in decreasing complexity, so can the input:

- **General points**: No restrictions on the points.
- **Distinctness assumption**: No 2 points have the same coordinates.

It turns out that all these variants have the same lower bound time complexity when restricted to algebraic decision tree algorithms in the RAM model (shown in section 3) and that there exists an algorithm matching this running time (shown in section 4.4). Accordingly, algorithms concerning convex hull computation from an asymptotic context, primarily deal with the convex hull sequence problem for general points. We approach the topic by considering general properties of the convex hull and their exploitation in algorithms.

There is a straightforward probabilistic version of the problem, where points are sampled from a (possibly known a priori) probability distribution. In section 5 we will show that for some distributions, the convex hull sequence problem can be solved in expected linear time with a simple heuristic.

In section 6 we present practical behavior of the algorithms in terms of comparisons performed, and verify that this matches the theoretical results. When implementing convex hull algorithms there are a number of issues related to hardware that need special concern, such as the floating point and overflow errors. In this thesis we will not delve into specifics of implementation matter. As such, our experimental work should be seen as exploratory and we warn the reader that the results should be taken with a grain of salt. For an in-depth experimental approach to convex hulls we refer to a paper by Gamby and Katajainen [6] that deals with these concerns in great detail.

The convex hull problem also exists for higher-dimensional spaces, but this will not be discussed in this paper.

[1] This notion of ordering problem complexity is defined more rigorously in section 3.2

3 Worst case time complexity lower bound

In this section we will show that the worst case time complexity for all convex hull problem variants described in section 2 is $\Omega(n \log h)$, where n is size of input and h is number of vertices in the convex hull, for all d-th order algebraic decision tree algorithms T, when d is constant. These include what can be referred to as comparison-based algorithms, such as the ones described in section 4.

3.1 Algebraic decision tree algorithms

An algebraic decision tree algorithm T of order d is represented by a rooted ternary tree. Leaves are labelled with a certain output. For the convex hull sequence problem this would be a permutation on a subset of the input, and for a decision problem such as the convex hull cardinality verification problem, leaves are labelled TRUE or FALSE. Each internal node is labelled with some multivariate polynomial of degree at most d. Each triplet of edges are labelled $<$, $=$ and $>$ respectively. See figure 1 for an example. The output of $T(\mathbf{x})$ on some multivariate input $\mathbf{x} \in \mathbb{R}^n$ is computed by traversing the simple path from root to leaf. On each node the algorithm computes the corresponding polynomial on $\mathbf{x}$, compares to 0, and follows the corresponding edge. When a leaf is reached, its value is returned. The worst case time complexity of T is at most its height. Note that decision problems can be stated in terms of an algebraic decision tree algorithms in the following way: identify the subset W of the input space that corresponds to TRUE output and construct a decision tree that *determine membership* in W. That is, $T(\mathbf{x}) = \text{TRUE}$ if and only if $\mathbf{x} \in W$. These types of decision trees will be of particular interest in the lower bound proof.

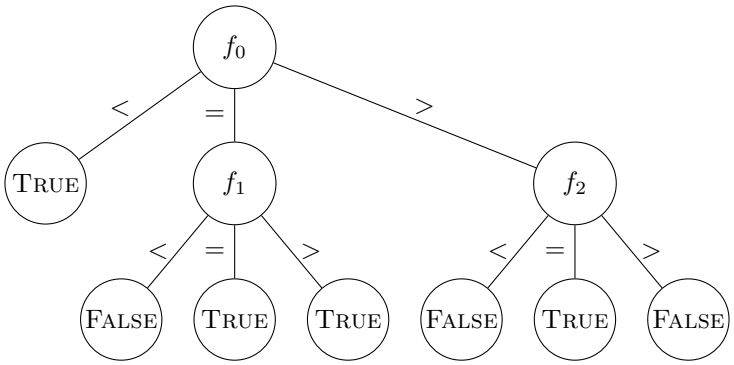


Figure 1: Example of algebraic decision tree. Polynomial f_i is of degree at most d.

3.2 Reduction proofs

Definition (Reduction)**.** Let $\mathbf{n}$ and $\mathbf{m}$ be the multivariate input sizes for problems A and B respectively. A reduction from B to A, is an implication stating that the existence of an algorithm solving A in time $f(\mathbf{n})$, implies the existence of an algorithm solving B in time $O(f(\mathbf{m})) + O(g(\mathbf{m}))$ where g is some amount of overhead. When $\Omega(h(\mathbf{m}))$ is a lower bound on B and $g(\mathbf{m}) = o(h(\mathbf{m}))$, we call it an *efficient* reduction (with respect to that lower bound). If an efficient reduction exists, a lower bound of $\Omega(h(\mathbf{n}))$ applies to A, we write $A \succeq B$ and say that B reduces to A.

Informally, problem B is *easier* than problem A in the sense that solving A also solves B. Soundness of this proof method is seen by contradiction; Say we have a lower bound

of $\Omega(h(\mathbf{m}))$ on problem B and an efficient reduction from B to A. Then assume for contradictory purposes that an algorithm that solves A in $o(h(\mathbf{n}))$ time exists. By the reduction, there then exists an algorithm solving B in $o(h(\mathbf{m})) + O(g(\mathbf{m}))$ time, arriving at a contradiction with the lower bound.

Reductions can naturally be extended into chains of reductions (i.e. transitivity) and are clearly reflexive. This motivates the notation of $A \succeq B$. We say that there is a reduction *from* B to A (perhaps alluding to the lower bound carrying over *from* B). The goal is to bring this reduction chain to a problem for which we can prove a lower bound directly. For this purpose, we introduce the multiset support cardinality verification problem (MS-verification).

3.3 Proving the lower bound of MS-verification

Definition (Multiset support cardinality verification problem, MS-verification)**.** Given a multiset M of n elements and an integer k, determine if M has exactly k distinct elements. In other words, the support of M has cardinality k.

To prove that the MS-verification problem has worst case time complexity $\Omega(n \log k)$ we will make use of the following theorem proven by Ben-Or [2, thm. 8].

Theorem 3.1 (Ben-Or)**.** Let $Y \in \mathbb{R}^N$ be any set and let T be any d'th order algebraic decision tree that determines membership in Y. If Y has M disjoint connected components, then T must have height, and hence worst case complexity $\Omega(\log M - N)$.

We will also need the very useful bound described below.

Theorem 3.2 (Stirling Bound)**.** For $n > 1$, we have $\log(n!) = \Omega(n \log n)$

Proof. This bound is a simple corollary of the broadly known Stirling's Approximation which states

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{\alpha_n}$$

for $n \geq 1$ and $\frac{1}{12n+1} < \alpha_n < \frac{1}{12n}$. Taking logarithms on both sides we get

$$\begin{aligned}
\ln(n!) &= \ln\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{\alpha_n}\right) \\
&= \ln\sqrt{2\pi n} + \ln\left(\left(\frac{n}{e}\right)^n\right) + \ln(e^{\alpha_n}) \\
&= \frac{1}{2}\ln(2\pi n) + n\ln\left(\frac{n}{e}\right) + \ln\alpha_n \\
&> \frac{1}{2}\ln(2\pi n) + n\ln\left(\frac{n}{e}\right) + \ln\left(\frac{1}{12n+1}\right) \\
&= \frac{1}{2}\ln(2\pi n) + n\ln n - n + \ln\left(\frac{1}{12n+1}\right) \\
&= \Omega(n\log n)
\end{aligned}$$

□

Now we are ready to prove the fundamental lower bound, that will be used in the subsequent reductions.

Theorem 3.3 (MS-verification Lower Bound)**.** The multiset size verification problem has worst case time complexity $\Omega(n \log k)$ for algebraic decision trees of fixed order.

Proof. The fundamental idea of this proof is taken from Kirkpatrick and Seidel [11, cor. 5.1], but we add a more comprehensive explanation.

View any problem instance M as a point $(m_1, m_2, \ldots, m_n) \in \mathbb{R}^n$. Then for a particular k, the problem space is $\mathbb{R}^n$. The crucial observation is that the problem space can be segmented into regions according to how many distinct coordinates a particular point has and solving MS-verification is then equivalent to determining membership in the following set which is the k-dimensional region of $\mathbb{R}^n$ where points have k distinct coordinates,

$$M_k = \{(m_1, m_2, \ldots, m_n) \in \mathbb{R}^n : |\{m_1, m_2, \ldots, m_n\}| = k\}$$

It suffices to show that this set has at least $k!k^{n-k}$ connected components (disjoint regions), as we then have,

$$\begin{aligned}
\text{Height}(T) &= \Omega(\log(k!k^{n-k}) - n) && \text{Ben-Or Theorem [3.1]} \\
&= \Omega(\log(k!) + \log(k^{n-k}) - n) \\
&= \Omega(k \log k + \log(k^{n-k}) - n) && \text{Stirling Bound [3.2]} \\
&= \Omega(k \log k + (n-k) \log k - n) \\
&= \Omega((k + (n-k)) \log k - n) \\
&= \Omega(n \log k - n) \\
&= \Omega(n \log k)
\end{aligned}$$

To realize that there are at least $k!k^{n-k}$ disjoint regions in M_k, consider the set of points in $\mathbb{R}^n$ created by setting the first k coordinates to a permutation of distinct coordinates $X = (x_1, x_2, \ldots, x_k) \in \mathbb{R}^k$ with $x_i < x_{i+1}$ and each of the remaining $n - k$ coordinates equal to one from X (as by replacement sampling from X). Clearly, these points all have k distinct coordinates, which means they are in M_k. Further, this subset of M_k defines a region and by permuting X and the replacement sampling we can obtain $k!k^{n-k}$ such regions, so it remains only to show that these regions are disjunct. For example in the case of $n = 3$, $k = 2$ these sets form $2!2^{3-2} = 4$ halfplanes,

$$\begin{gathered}
(u, v) \in \mathbb{R}^2, \quad u < v \\
a : (u, v, u) \quad b : (v, u, v) \\
c : (u, v, v) \quad d : (v, u, u)
\end{gathered}$$

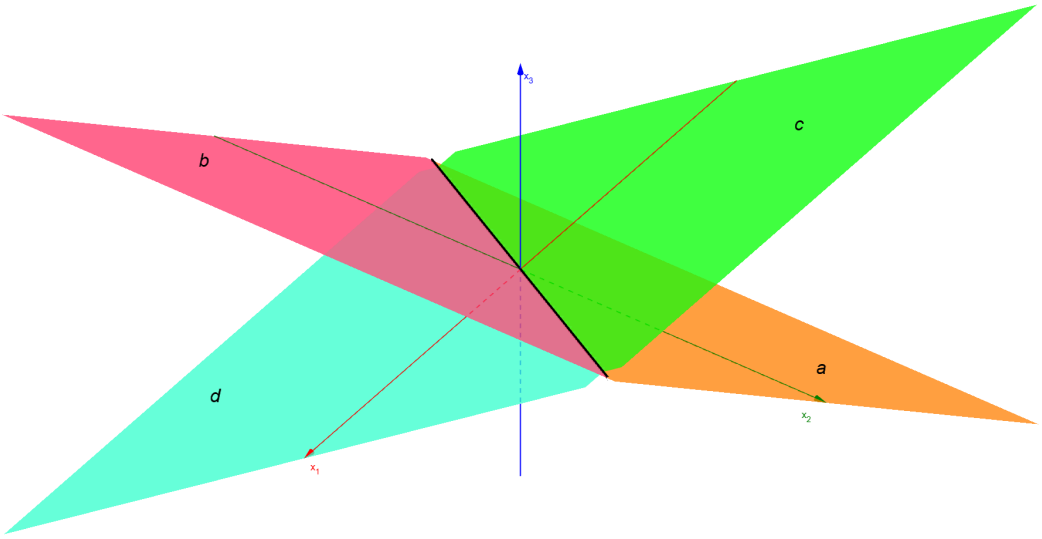


Figure 2: The 4 regions of M_k for $n = 3$, $k = 2$.

In general these sets define regions of k-dimensional hyperplanes in the n-dimensional space. In order to prove that each region is disjunct from the other regions, we show that there exists no continous path between any of these regions that is contained in M_k. The argument is intuitive to follow for the above example, but does not depend on 3 dimensions so it generalizes to any combination of n and k (obviously $k \leq n$ must hold). Notice that in order to go from any region to another the path must necessarily go through the space surrounding the planes or go through their division line (the black line in figure 2). If the path goes through the surrounding space, the number of distinct coordinates will increase. If the path goes through the division line, the number of distinct coordinates will decrease. Either way, the number of different coordinates is different from k in these cases and thus not in M_k. □

3.4 Proving the reduction from MS-verification

Recall that a reduction to A from B means showing an implication from the existence of an algorithm solving A to the existence of an algorithm solving B with some overhead. The lower bound on MS-verification being $\Omega(n \log k)$ means we can allow $o(n \log k)$ overhead when creating reductions, but in practice $O(n)$ overhead will be enough. One way of actually proving reductions are by defining a transformation f on the input space of B that takes it to the input space of A and a transformation g on the output space of A that takes it back to the output space of B such that correctness carries over to the new problem ("correspondence"). If g and f are $O(n)$ this is a valid reduction since by assuming an algorithm M that solves A we can compose $g \circ M \circ f$ to solve B.

For a simple example of reductions among the convex hull variants, the convex hull sequence problem (CH-sequence) can be reduced from the convex hull cardinality verification problem (CH-verification). The input space is the same, and output correspondence can be created by counting elements of the outputted CHV(Q) and comparing to k. This particular transformation is linear in h which is $O(n)$. Similarly, any algorithm for general points can solve the same problem with the distinctness assumption with no transformation overhead simply by applying the same algorithm. The reader can easily verify that the

remaining variants of convex hull given in section 2 reduce from CH-verification under the distinctness assumption.

Proving reductions to CH-verification under distinctness assumption from MS-verification is more complicated, in order to bridge the gap we introduce some new definitions and problems. Our proof is very inspired by Kirkpatrick and Seidel [11, lem. 5.1-2] but more thorough. An input instance to CH-verification can be written as $Q = [q_1, \ldots, q_n] = [(x_1, y_1), \ldots, (x_n, y_n)]$. Let $\overline{q} = \left(\frac{1}{n}\sum_{i=1}^{n} x_i, \frac{1}{n}\sum_{i=1}^{n} y_i\right)$ denote the center of Q. Multiset Q is called center-free if and only if no point in Q lies on the center. Let ϵ be some small positive value. We introduce two sets:

$$C_h = \{Q : |\mathrm{CHV}(Q)| = h\}$$
$$P_h = \{Q : |\mathrm{CHV}(\{q_i + i(q_i - \overline{q})\epsilon : 1 \leq i \leq n\})| = h\}$$

Set C_h contains all multisets with h vertices in their convex hull polygon. Set P_h contains all multisets Q with h vertices in their convex hull polygon after moving each point a tiny bit unevenly away from the center (this is illustrated in figure 3).

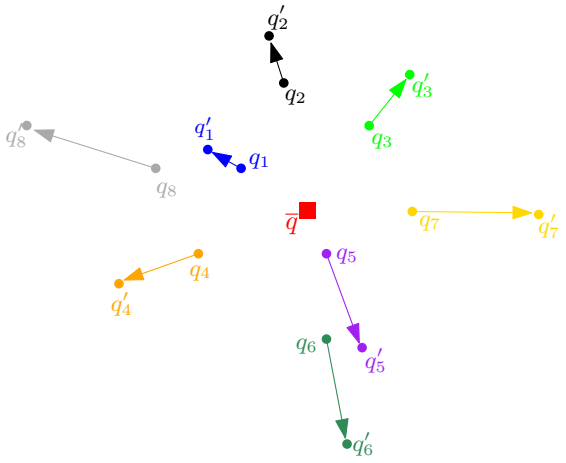


Figure 3: Illustration of transforming each point $q_i \in Q$ into $q'_i = q_i + i(q_i - \overline{q})\epsilon$

Let Member(S) be the problem of determining membership in a set S. Notice that Member(C_h) and Member(M_k) are equivalent to solving CH-verification and MS-verification respectively. Throughout this proof we will show the following,

$$\begin{aligned}
&\text{any CH problem} \\
\succeq\ &\text{CH-verification} && \text{(distinctness assumption)} \\
\approx\ &\text{Member}(C_h) && \text{(distinctness assumption)} \\
\succeq\ &\text{Member}(P_h) && \text{(center-free assumption)} \\
\succeq\ &\text{Member}(M_k) \\
\approx\ &\text{MS-verification}
\end{aligned}$$

By transitivity, the above reduction chain shows that all the CH problems reduce from MS-verification, which lower bounds them. The equivalences ($\approx$) and the first reduction (any CH problem $\succeq$ CH-verification) are easy to realize. Completing the proof we have to show that the two last reductions in the above reduction chain holds. Showing that Member(C_h) $\succeq$ Member(P_h) we use the following lemma.

Lemma 3.3.1. Let T be any d'th order decision tree algorithm for deciding membership in C_h, where input is assumed distinct. Then there exists a d'th order decision tree algorithm T' of same worst case asymptotic complexity that decides membership in P_h where input is assumed to be center-free, but does not have the distinctness assumption. That is, Member(C_h) under the distinctness assumption reduces from Member(P_h) under the center-free assumption.

Proof. Consider the input space for T'. Any instance is some center-free multiset $Q = [q_1, q_2, \ldots, q_n]$. First let

$$Q' = [q_i + i(q_i - \bar{q})\epsilon : 1 \leq i \leq n]$$

Observe that with sufficiently small ϵ, Q being center-free implies distinctness in Q'. This transformation also maintains $Q' \in C_H \Leftrightarrow Q \in P_H$. So $T(Q')$ correctly determines membership of Q' in C_H, which in turn determines membership of Q in P_H. We want to devise a T' such that $T'(Q) = T(Q')$, since this solves the desired membership problem.

Consider any non-leaf vertex of T, which defines a subtree rooted in some v_j with label $f(Q)$. Then define d multivariate polynomials by the equality

$$f_j(Q') = f_{j,0}(Q) + f_{j,1}(Q)\epsilon + \ldots + f_{j,d}(Q)\epsilon^d \qquad \text{(Eq. 3.1)}$$

Observe that no matter what polynomial f_j is, expanding it out and regrouping by ϵ we see that the degree of each $f_{j,i}$ is at most d. We use this to define T' as a transformation of T. Leafs remain unchanged under the transformation while the subtree rooted in some v_j is given by figure 4. It is quite easily seen that the longest path in T' is going through all $f_{j,i}$ and then selecting the highest subtree (repeatedly), resulting in a path that is $(d+1)$Height(T) long. Since d is fixed T' has the same worst case asymptotic complexity.

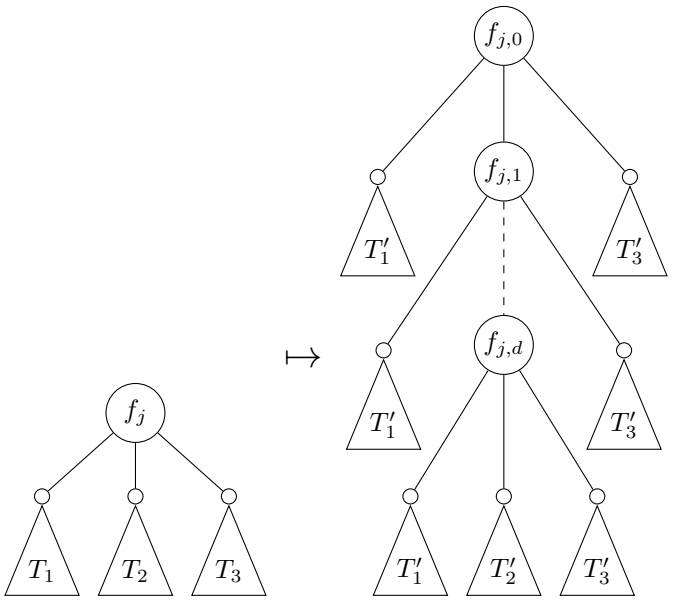


Figure 4: Transformation on subtree rooted at note v_j

It remains only to show that the transformed subtree on Q (right on figure 4) agrees with the original subtree on Q' (left on figure 4). In this effort, the critical observation is that for sufficiently small ϵ the term $f_{j,i}(Q)\epsilon^i$ in Eq. 3.1 will dominate $f_{j,i+1}(Q)\epsilon^{i+1}$. That is,

the earlier nonzero terms in Eq. 3.1 can be made arbitrarily larger than later terms by decreasing ϵ. This means that if $f_j(Q')$ is positive, then $f_{j,0}(Q)$ will have to be nonnegative for Eq. 3.1 to hold. If $f_{j,0}(Q)$ is positive the subtrees agree, and if $f_{j,0}(Q)$ is zero, the same argument can be applied inductively down to $f_{j,d}(Q)$ which then has to be positive for Eq. 3.1 to hold. A symmetric argument applies if $f_j(Q')$ is negative. If $f_j(Q')$ is zero, no $f_{j,i}(Q)$ can be nonzero since a single nonzero term could not be cancelled out by other factors in Eq. 3.1. All in all, the transformed subtree on Q' will agree with the original subtree on Q and by extension $T(Q') = T'(Q)$.

□

Lemma 3.3.2. Determining membership in P_h with the center-free assumption reduces from determining membership in M_k.

Proof. For any problem instance M of determining membership in M_k, transform the input as follows

$$M = [x_1, \ldots, x_n] \mapsto Q = [(x_1, x_1^2), \ldots (x_n, x_n^2)]$$

The degenerate case of all multiplicities can be solved in linear time, otherwise note that Q lie on a parabola which is clearly center-free, so we can use the algorithm for determining membership in P_h. Further, the only points in Q that are not in $\mathrm{CHV}(Q)$ are multiplicities. Since the transformation preserves the number of multiplicities, setting $h = k$ there is correspondence in output,

$$Q \in P_h \Leftrightarrow |\mathrm{CHV}(Q)| = h \Leftrightarrow M \in M_k$$

□

Theorem 3.4. All the problems in section 2 have lower bound $\Omega(n \log h)$, with any d'th order decision tree algorithm (which includes comparison-based algorithms).

Proof. Follows directly from the lower bound on the multiset cardinality verification problem of theorem 3.3 and the reduction chain in section 3.4. □

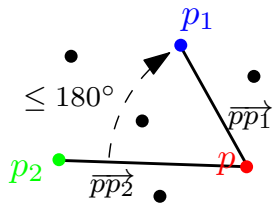
4 Algorithms

In this section we describe and analyse three convex hull algorithms. The analysis of each algorithm includes the analysis of the asymptotic running time and correctness.

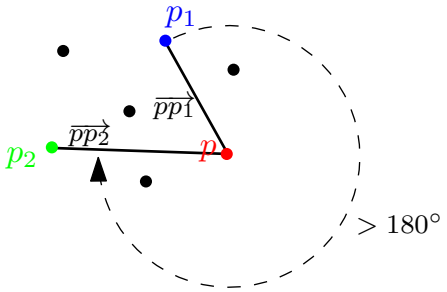
4.1 Fundamentals

The convex hull sequence problem is intentionally worded without defining a specific extremal point. This is due to the very simple property of the convex hull, that the extremal point in any direction is in $\mathrm{CHV}(Q)$[2]. If multiple extremal exist in one direction, the perpendicular direction must then be considered secondarily. In this paper we will use the point with least y-coordinate, ordering secondarily by least x-coordinate. This choice is rather arbitrary and all algorithms in this section can easily be reformulated to comply with other choices. Likewise for the direction of the cyclic sequence, we arbitrarily choose counterclockwise direction. The process of finding this first point is the starting point of all algorithms in this section and we will denote it by Bottom-Left(Q), it is easily seen that this can be implemented in linear time.

A useful comparison operator for convex hull algorithms is the orientation test: Given an origin point p, the orientation test between two points p_1 and p_2, determines if the directed segment $\overrightarrow{pp_1}$ is closer to $\overrightarrow{pp_2}$ by clockwise or counterclockwise rotation. We write infix $p_1 \circlearrowright_p p_2$ and say that p_1 is more clockwise than p_2 (with respect to p) if $\overrightarrow{pp_1}$ is within an 180 degree angle clockwise from $\overrightarrow{pp_2}$. See figure 5 for an example.



(a) The point p_1 is more clockwise than p_2 with respect to p (within 180° clockwise). In this case $p_1 \circlearrowright_p p_2$ is True



(b) The point p_2 is not more clockwise than p_1 with respect to p (not within 180° clockwise). In this case $p_2 \circlearrowright_p p_1$ is False

Figure 5: Example of the orientation test

The orientation test operator can be implemented efficiently by computing the cross product and then comparing with 0:

$$\begin{aligned} p_1 \circlearrowright_p p_2 &\Leftrightarrow (p_1 - p) \times (p_2 - p) \geq 0 \\ &\Leftrightarrow \det \begin{pmatrix} x_1 - x_0 & y_2 - y_0 \\ y_1 - y_0 & x_2 - x_0 \end{pmatrix} \geq 0 \\ &\Leftrightarrow (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0) \geq 0 \end{aligned}$$

In many senses $\circlearrowright_p$ mimics the ordering properties of $\geq$. Accordingly, the orientation test can be used to define $\mathrm{cw}_p(Q)$ which returns the most clockwise point of multiset $Q \subset \mathbb{R}^2$, just as $\max(A)$ would return the largest number of multiset $A \subset \mathbb{R}$. There

[2] In section 5.1 we investigate this property more in depth

are some considerations to make this definition well-defined; $\text{cw}_p(Q)$ is only well-defined if there exists an open half-plane through p that contains Q as otherwise the sequence is not monotonic. See figure 6. The only time we will use this procedure is if $p \in \text{CHV}(Q)$, and then the only point not contained in the half-plane is exactly p. We resolve this by returning $\text{cw}_p(Q - \{p\})$. Furthermore, even if Q contains no duplicates, $\text{cw}_p(Q)$ might not be unique, this happens if there are two points that are collinear with p. For our usecase it will convenient to resolve this by ordering secondarily by largest distance from p. In general, whenever an algorithm finds collinear points, such as with the orientation test, it will always exclude the middle points, as these cannot be vertices of the convex hull. We will use MOST-CLOCKWISE-FROM(p, Q) to denote an algorithm that implements these considerations, which can clearly be done in linear time. Analogous definitions and properties apply for the counterclockwise direction $\circlearrowleft_p$, $\text{ccw}_p(Q)$ and MOST-COUNTERCLOCKWISE-FROM(p, Q). Theorem 4.1 shows a strong property of MOST-CLOCKWISE-FROM.

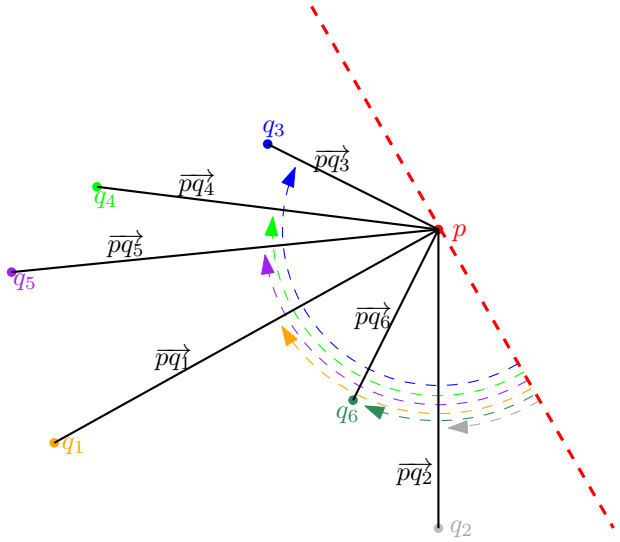


Figure 6: An illustration of $\text{cw}_p(Q)$ with the origin point p and the set $Q = \{q_1, ..., q_6\}$. We see that $\text{cw}_p(Q) = q_3$. The red dashed line represents a possible half-plane that contains Q under it, making the result well-defined.

Theorem 4.1. Given some point $p \in Q$ that is also in $\text{CHV}(Q)$, we have that MOST-CLOCKWISE-FROM(p, Q) is the next vertex of the cyclically ordered convex hull.

Proof. First of all, MOST-CLOCKWISE-FROM(p, Q) is well-defined since $p \in \text{CHV}(Q)$. Assume for contradictory purposes that MOST-CLOCKWISE-FROM(p, Q) is not the next point of the cyclically ordered convex hull. Clearly, there is some other point $p' \in Q$ which is, but any sequence of points from p through p' will then eventually include a right turn in order to contain MOST-CLOCKWISE-FROM(p, Q), which is a contradiction since then the hull is not convex. □

4.2 Graham's scan

The Graham's scan algorithm was first introduced by Ronald L. Graham in an article in 1972 [7]. The algorithm is one of the most famous, if not the most famous, convex hull algorithm and it highlights the similarities between sorting and the convex hull problem.

Since Ronald L. Graham introduced the algorithm, it has been improved by changing certain operations, such as the way of calculating the polar angles of each point. Therefore, we will base our portrayal in the version from CLRS [4, p. 1031]. Note that our Python-inspired notation is different from CLRS in that Q'_i denotes the i'th point in Q', while $Q'_{1..i}$ denotes the subsequnce of the first i points. We write S_{-1} to mean the last element of S, i.e. the top of the stack.

The pseudocode of Graham's scan is presented below as GRAHAM-SCAN. First the algorithm finds the extremal point p_1 that is the first point in CHV(Q). Next the remaining points are sorted by their polar angle relatively to p_1. This can be done with any comparison sort using the $\circlearrowleft_{p_1}$ operator under the same considerations as MOST-CLOCKWISE-FROM laid out in section 4.1.

The rest of the algorithm is a linear scan through the sorted points, maintaining the stack S. The scan builds on an alternative interpretation of the orientation test which is that visiting three points in order can form a left turn, right turn or a straight line (collinear points). This can be implemented efficiently by realizing that "(p_1, p_2, p_3) forms a nonleft[3] turn" is equivalent to $p_1 \circlearrowleft_{p_2} p_3$. Finally, when the scan terminates, the stack S is returned containing CHV(Q) in cyclical order.

Algorithm 1 GRAHAM-SCAN(Q)

```
1:  let p_1 = BOTTOM-LEFT(Q)
2:  let Q' be the other points in Q, sorted in counterclockwise order around p_1
3:  remove all points in Q' that are collinear with p_1 except the one furthest away
4:  prepend p_1 to Q'
5:  let S be an empty stack
6:  PUSH(Q'_1, S)
7:  PUSH(Q'_2, S)
8:  PUSH(Q'_3, S)
9:  for i = 4 to |Q'| do
10:     while (S_{-2}, S_{-1}, Q'_i) forms a non-left turn do
11:         POP(S)
12:     PUSH(Q'_i, S)
13: return S as a sequence from bottom to top
```

Theorem 4.2 (Time complexity of GRAHAM-SCAN)**.** GRAHAM-SCAN has a worst case time complexity of $O(n \log n)$

Proof. Analysing the time complexity, the lines in GRAHAM-SCAN are analysed separately and lastly the accumulated running time is derived. Finding p_1 on line 1 takes $c_1 n$ steps worst case. Sorting the remaining points by polar angle in counterclockwise order, on line 2, takes $c_2 n \log n$ steps using any $O(n \log n)$ comparison sort algorithm (eg. HEAP-SORT or MERGE-SORT). Line 3 can be performed in $c_3 n$ steps with a linear traversal, by realizing that all collinear points with p_1 are neighbors after sorting. Prepending p_1 and initializing S on lines 4-8, takes c_4 steps. The operations on line 10-12 are an example of the classic *stack with multipop* problem used to teach introductory amortized analysis (see [4, p. 452]), and takes $c_5 n$ steps worst case. Finally, it takes c_6 steps to return the stack S on line 13.

[3] Nonleft means right or straight

Adding the running times analysed above we get a running time,

$$T(\textsc{Graham-Scan}) = c_1 n + c_2 n \log n + c_3 n + c_4 + c_5 n + c_6 = O(n \log n)$$

□

Note that the running time is dominated by the sorting algorithm used, so given information that can be exploited during sorting, this algorithm is potentially even faster.

Theorem 4.3 (Correctness of Graham-Scan)**.** Graham-Scan solves the convex hull sequence problem.

Proof. This correctness proof is strongly inspired by the proof derived by CLRS [4, p. 1034]. After line 4 in Graham-Scan has terminated, we have a sequence Q'. Note that that $Q - Q'$ are the points that were collinear with some other point in Q relatively to p_1 and the closest to p_1 of the two. We have that $p \notin \text{CHV}(Q)$ for all $p \in Q - Q'$. Thus, $\text{CHV}(Q') = \text{CHV}(Q)$, and therefore it suffices to show that S contains $\text{CHV}(Q')$ in cyclical order.

The proof uses the following loop invariant:

> At the start of each iteration of the for-loop on line 9, the stack S consists of $\text{CHV}(Q'_{1..i-1})$ in counterclockwise order from bottom to top

Initialization: From lines 4-8 we have that $S = (Q'_1, Q'_2, Q'_3)$. Clearly $Q'_1, Q'_2, Q'_3 \in \text{CHV}(Q'_{1..3})$. We know that $Q'_1 = p_1 = \text{CHV}(Q'_{1..3})_1$, since it is has the lowest y-coordinate of all points in Q, or the leftmost such point in case of a tie. By theorem 4.1 we also know that $p_2 = \text{CHV}(Q'_{1..3})_2$. Since only Q'_3 is left to consider, we must have $Q_3 = \text{CHV}(Q'_{1..3})_3$. We have that $i = 4$ at the first iteration, which means that the invariant holds at the beginning of the first iteration.

Maintenance: At the start of iteration i of the for-loop, we have that Q'_{i-1} is the top element in the stack S, since it was pushed on line 12 in iteration $i-1$. Let Q'_j be the top element of S after the while-loop has terminated, but before pushing Q'_i. By the loop invariant we have that S contains the exact vertices of $\text{CHV}(Q'_{1..j})$. This holds since all points pushed on the stack after the j'th iteration has been popped and therefore the stack looks exactly as it did at the start of iteration $j+1$. We know that (Q'_{j-1}, Q'_j, Q'_i) makes a left turn, since Q'_j would have been popped in the while-loop otherwise. This means that when Q'_i is pushed onto S on line 12, we have that S contains exactly $\text{CHV}(Q'_{1..j}) \cup \{Q'_i\} = \text{CHV}(Q'_{1..j} \cup \{Q'_i\})$, which holds by the symmetric version of theorem 4.1 because Q'_i is more counterclockwise from Q'_1 than any element in $\text{CHV}(Q'_{1..j})$.

Now we remain to show that $\text{CHV}(Q'_{1..j} \cup \{Q'_i\}) = \text{CHV}(Q'_{1..i})$. Let Q'_t be any point that was popped during iteration i. We have that (Q'_{t-1}, Q'_t, Q'_i) makes a non-left turn, since Q'_t was popped and we also know that Q'_t is more counterclockwise than Q'_{t-1} relatively to Q'_1. This means that Q'_t must be on the interior of the triangle formed by $\{Q'_1, Q'_{t-1}, Q'_i\}$ as shown in Figure 7. Therefore $Q'_t \notin \text{CHV}(Q'_{1..i})$, since it is not even in $\text{CHV}(\{Q'_1, Q'_{t-1}, Q'_t, Q'_i\})$. Thus, we have that $\text{CHV}(Q'_{1..i} - \{Q'_t\}) = \text{CHV}(Q'_{1..i})$ for any Q'_t popped in the while-loop, which means that $\text{CHV}(Q'_{1..i}) = \text{CHV}(Q'_{1..j} \cup \{Q'_i\})$. Conclusively, we have that after incrementing i, the loop invariant still holds because $\text{CHV}(Q'_{1..j}) \cup \{Q'_i\} = \text{CHV}(Q'_{1..j} \cup \{Q'_i\}) = \text{CHV}(Q'_{1..i})$, which is represented in the stack S in counterclockwise order from bottom to top.

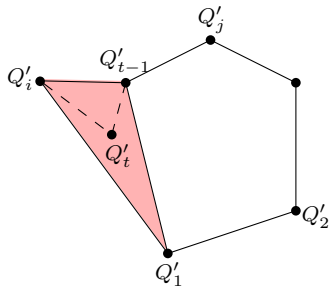


Figure 7: Q_t is on the interior of the triangle formed by $\{Q'_1, Q'_{t-1}, Q'_i\}$, since (Q'_{j-1}, Q'_j, Q'_i) makes a non-left turn

Termination: When the for-loop terminates, we have that $i = |Q'| + 1$, which by the loop invariant means that $\text{CHV}(Q'_{1..i-1}) = \text{CHV}(Q'_{1..|Q'|})$. By definition, $Q'_{1..|Q'|} = Q'$ and in the start of the proof we showed that $\text{CHV}(Q') = \text{CHV}(Q)$. □

4.3 Jarvis' march

The Jarvis' march algorithm, also known as the gift wrapping algorithm, was first introduced by Ray A. Jarvis in 1973 [9]. Jarvis' march is extremely simple and works by continuously applying theorem 4.1.

First the algorithm finds the extremal point p_1 that is the first point in $\text{CHV}(Q)$. An empty sequence H is initialized, which will end out as $\text{CHV}(Q)$ in counterclockwise cyclical order. An iterating point p' is initialized as p_1 which is the first point of $\text{CHV}(Q)$. In a loop p' is continuously updated to be the next point of $\text{CHV}(Q)$ by Most-Clockwise-From and added to the sequence until all h vertices are found.

Algorithm 2 Jarvis-March(Q)

```
1: let p_1 = Bottom-Left(Q)
2: let H be an empty sequence
3: let p' = p_1
4: repeat
5:     append p' to H
6:     let p' = Most-Clockwise-From(p', Q)
7: until p' = p_1
8: return H
```

Theorem 4.4 (Time complexity of Jarvis-March)**.** Jarvis-March has a worst case time complexity of $O(nh)$

Proof. Analysing the time complexity, the lines in Jarvis-March are analysed separately and lastly the accumulated running time is derived. Finding p_1 on line 1 takes $c_1 n$ steps worst case. Defining the values on line 2-3 takes c_2 steps. Inside the repeat-until loop on lines 4-7 we add p' to the stack H in c_3 steps. Then we update p' by calling Most-Clockwise-From(p, Q) which takes $c_4 n$ steps. The repeat-until loop iterates exactly h times (see correctness proof) and therefore we have that the lines 4-7 take $h(c_3 + c_4 n)$ steps. Finally, H is returned in c_5 steps.

Adding the running times analysed above, we get that JARVIS-MARCH has a running time of:

$$T(\text{JARVIS-MARCH}) = c_1 n + c_2 + h(c_3 + c_4 n) + c_5 = O(nh)$$

Conclusively, we see that JARVIS-MARCH has relatively good time complexity when h is small. □

Theorem 4.5 (Correctness of JARVIS-MARCH)**.** JARVIS-MARCH solves the convex hull sequence problem.

Proof. Correctness follows directly from the following loop invariant:

> After line 5 of iteration i of the repeat loop, the sequence H consists of the first i points of CHV(Q) and p' is the latest point.

Initialization: Line 3 sets p' to p_1, which is always the first point in CHV(Q). Line 5 adds this point to H, completing the initialization for iteration 1.

Maintenance: In iteration i, by the loop invariant, we assume that H consists of the first i points of CHV(Q) and p is the i'th point. Line 6 sets p' to the most clockwise point in Q with respect to point i of CHV(Q), which by theorem 4.1 is the next point in CHV(Q). Line 5 adds the point to H, maintaining the invariant for iteration $i + 1$.

Termination: In iteration h of the loop, line 6 finds the next point CHV(Q), but since CHV(Q) is cyclic this is equal to p_1, terminating the loop.

□

4.4 Chan's algorithm

CHANS-ALGORITHM was first introduced by Timothy M. Chan in 1996 [3]. CHANS-ALGORITHM is the only time-optimal algorithm discussed in this paper. It combines GRAHAM-SCAN and a variant of JARVIS-MARCH in a divide-and-conquer fashion, along with a clever parameter search to achieve time-optimality. It is not the first time-optimal convex hull algorithm (see [11]) but arguably the simplest. The algorithm described below is similar to the one described in Timothy M. Chan's original article [3], but formulated to be consistent with our version of JARVIS-MARCH. We also include a description of the central binary search principle that makes the algorithm possible, which we were unable to find elsewhere.

The fundamental idea of CHANS-ALGORITHM is to partition Q into arbitrary equal-sized subsets and use GRAHAM-SCAN to compute the convex hull of each subsets ("subhull"). Then these subhulls are combined by a variant of JARVIS-MARCH that in each iteration exploits convexity to first compute MOST-CLOCKWISE-FROM of each subhull using binary search. The most clockwise point of each subhull is combined with a regular call to MOST-CLOCKWISE-FROM, finding the most clockwise point in general.

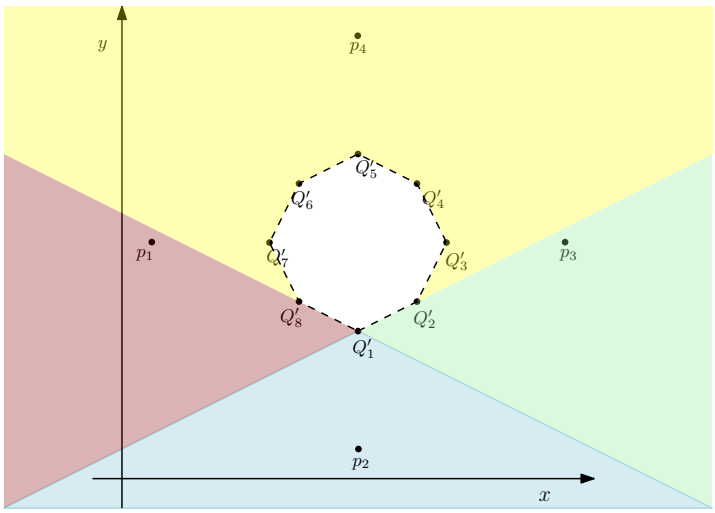
Theorem 4.6 (Binary Search on Convex Hulls)**.** Given a cyclically ordered convex hull C of k points and a point p outside or on a vertex of C, we have that MOST-CLOCKWISE-FROM(p, C) can be computed in $O(\log k)$ steps.

Proof. Consider the order imposed by $\circlearrowleft_p$ and its similarities to $\leq$ as described in section

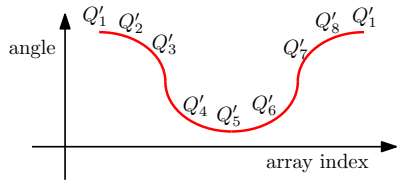
4.1. It becomes clear that C can be represented by an array of numbers (angles from p) and the problem is then equivalent to finding the maximum value. This array representation is cyclic in the same way that C is, so the array represents an entire period. From the convexity of C we can reason that the cyclic array (wrapping edges together) has only one monotonically increasing part and only one monotonically decreasing part. So finding the maximum is equivalent to finding the the peak (element with smaller neighbors). A corresponding visual, is a plot with points in C along the first axis with their array value along the second axis, the points will lay on a sine-like wave. Note however that the starting point of C is arbitrary and has no connection to finding the maximum and we thus have no knowledge of the phase of the wave. This means that we cannot do regular binary search on a single period. Instead we must consider 4 cases, illustrated in figure 8. The simplest case is that the period begins with the monotonic decrease (b in figure 8), in this case the edge is the maximum. Another simple case is that the period begins with the monotonic increase (c in figure 8), in which case we can do regular binary search. The tricky case, is when the period begins during the monotone increase or decrease (d and e in figure 8). In this case the monotone part is broken into two, creating a "fake peak" on one edge that is not the maximum (and a "fake valley" on the other side). To solve this we start both a binary search for the maximum and minimum element (minimum corresponds to Most-Counterclockwise-From). It is possible that one of these end up in the fake peak or valley which is already known, but they cannot both. We can detect which result avoided the fake peak/valley. If we found a new peak this must be the maximum. If we found a new valley we can do binary search on the part of the array between the fake valley and the newly found valley, which must contain the maximum. Detecting which case is relevant can be done in constant time by orientation test on the edges of the period and the worst case (d and e) still only performs three runs of binary search so it can be done in $O(\log k)$.

□

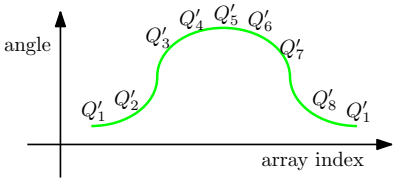
Other than the multiset of points Q, Chans-Algorithm also takes two other parameters. The parameter m is the size of the subsets created on line 2 and the parameter h' is an estimate of h and is used to bound the time spent doing Jarvis-March, which is necessary to achieve time-optimality. If h' is large enough, then the algorithm will return the convex hull. In the case that h' is not large enough, the algorithm will eventually terminate and return Incomplete.



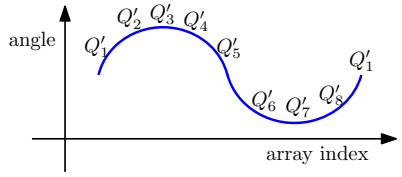
(a) Each colored region represents one of the 4 cases shown in figure b-e. Note that the indices in the below graphs are cyclic, which is why Q'_1 is the first and last index



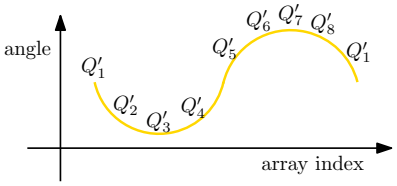
(b) Angles of $Q'_1, ..., Q'_8$ in relation to p_1



(c) Angles of $Q'_1, ..., Q'_8$ in relation to p_3



(d) Angles of $Q'_1, ..., Q'_8$ in relation to p_2



(e) Angles of $Q'_1, ..., Q'_8$ in relation to p_4

Figure 8: Example of convex polygon and 4 cases, represented by the points p_1, p_2, p_3, p_4 in relation to the convex polygon $Q'_1, ..., Q'_8$

Algorithm 3 CHANS-ALGORITHM(Q, m, h')

let $p_1 =$ BOTTOM-LEFT(Q)
let $Q_1, Q_2, ..., Q_{\lceil \frac{n}{m} \rceil}$ be an arbitrary partition of Q into subsets of size at most m
for $i = 1$ **to** $\lceil \frac{n}{m} \rceil$ **do**
    let $S_i =$ GRAHAM-SCAN(Q_i)
let H be a sequence containing p_1
let $p' = p_1$
for $k = 1$ **to** h' **do**
    for $i = 1$ **to** $\lceil \frac{n}{m} \rceil$ **do**
        let $s_i =$ MOST-CLOCKWISE-FROM(p', S_i) computed by binary search
    let $p' =$ MOST-CLOCKWISE-FROM$(p', \{s_1, \ldots, s_{\lceil \frac{n}{m} \rceil}\})$
    if $p' = p_1$ **then**
        return H
    else
        append p' to H
return INCOMPLETE

In theorem 4.7 we show that the time complexity of CHANS-ALGORITHM is $O(n \log m)$ when $m = h'$. We want to choose h' such that the for-loop on line 7 does not terminate before the algorithm goes into the if-statement on line 11. On the other hand, we want the m and h' to be as small as possible. In fact, we want that $h' = h$, since this is the exact number of iterations the for-loop on lines 7-14 needs to get into the if-statement on line 11. Also, we want that $h' = m$, since it causes the for-loop on lines 3-4 and the for-loop on lines 7-14 to both have a running time of $O(n \log m)$, which is optimal because there is a trade-off between the two. As proved in section 3, the convex hull cardinality problem has a time complexity of $\Omega(n \log h)$, which means that it is expensive to find h. One way to guarantee running time of $O(n \log h)$ without knowledge of input while preserving correctness, is by using the SQUARING-STRATEGY algorithm shown below to approximate h. SQUARING-STRATEGY iterates over a counter t. Each iteration m and h' are set to 2^{2^t}. CHANS-ALGORITHM(Q, m, h') is run at each iteration until $h' = 2^{2^t} \geq h$, since CHANS-ALGORITHM is guaranteed to return a convex hull with that parameter.

Algorithm 4 SQUARING-STRATEGY(Q)

```
1: for t = 1, 2, ... do
2:     m = h' = min(2^(2^t), n)
3:     L = CHANS-ALGORITHM(Q, m, h')
4:     if L ≠ INCOMPLETE then
5:         return L
```

Theorem 4.7 (Time complexity of CHANS-ALGORITHM)**.** CHANS-ALGORITHM has a worst case time complexity of $O(n \log m)$ when $m = h'$

Proof. Finding p_1 on line 1 takes $c_1 n$ steps worst case. Partitioning Q into $\lceil \frac{n}{m} \rceil$ arbitrary subsets takes $c_2 n$ steps. As proved in theorem 4.2, GRAHAM-SCAN takes $O(n \log n)$ steps. Each subset Q_i has a size of at most m, which means that GRAHAM-SCAN(Q_i) takes $O(m \log m)$ steps. Thus, the for-loop on line 3 takes $c_3 \lceil \frac{n}{m} \rceil \cdot m \log m =^4 c_3 \frac{n}{m} m \log m = c_3 n \log m$ steps. Initializing H and p' on lines 5-6 takes c_4 steps. The for-loop on lines 7-14 iterates $h' = m$ times. Performing binary search on each subset S_i takes $O(\log m)$ steps (because $|S_i| \leq |\text{CHV}(Q_i)| \leq m$), which means that the nested for-loop on lines 8-9 take $c_5 \lceil \frac{n}{m} \rceil \log m = c_5 \frac{n}{m} \log m$ steps. Line 10 takes $c_6 \frac{n}{m}$ steps. Clearly, the worst case is that line 12 is never reached, so the other branch must be taken. Lines 13-14 take c_7 steps. Lastly, line 15 takes c_8 steps. Adding the running times,

$$
\begin{aligned}
&T(\text{CHANS-ALGORITHM with } m = h') \\
&= c_1 n + c_2 n + c_3 n \log m + c_4 + m(c_5 \frac{n}{m} \cdot \log m + c_6 \frac{n}{m} + c_7) + c_8 \\
&= c_1 n + c_2 n + c_3 n \log m + c_4 + c_5 n \log m + c_6 n + m c_7 + c_8 \\
&= O(n \log m)
\end{aligned}
$$

□

[4] c_3 in $c_3 \lceil \frac{n}{m} \rceil \cdot m \log m$ is different from c_3 in $c_3 \frac{n}{m} \cdot m \log m$, even though we have written that the terms are equal. This is ignored since it is only important that c_3 is a constant. We do the same in the rest of the analysis

Theorem 4.8 (Time complexity of CHANS-ALGORITHM under SQUARING-STRATEGY)**.** CHANS-ALGORITHM has a worst case time complexity of $O(n \log h)$ under SQUARING-STRATEGY.

Proof. As explained earlier, the for-loop in SQUARING-STRATEGY iterates until $h' = 2^{2^t} \geq h \Leftrightarrow t \geq \log\log h$. In fact there are exactly $t = \lceil \log\log h \rceil$ iterations, since this is the exact iteration where it happens that $h' \geq h$. In the case where $h' = m = n$, the CHANS-ALGORITHM will return CHV(Q), since $h \leq n$. All operations inside the for-loop takes constant time except line 3 where CHANS-ALGORITHM is executed. As proved above, CHANS-ALGORITHM has a running time of $O(n \log m)$, which means that line 3 takes $c_t n \log 2^{2^t} = c_t n 2^t$ steps at iteration t. Thus, we get a running time of:

$$
\begin{aligned}
& T(\text{CHANS-ALGORITHM under SQUARING-STRATEGY}) \\
&= \sum_{t=1}^{\lceil \log\log h \rceil} c_t n 2^t \\
&\leq \max(c_1, ..., c_{\lceil \log\log h \rceil}) n \sum_{t=1}^{\lceil \log\log h \rceil} 2^t && \text{Distributivity} \\
&= \max(c_1, ..., c_{\lceil \log\log h \rceil}) n \frac{2^{\lceil \log\log h \rceil + 1} - 2}{2 - 1} && \text{Geometric series} \\
&= \max(c_1, ..., c_{\lceil \log\log h \rceil}) n \cdot 2 \cdot 2^{\lceil \log\log h \rceil} - 2 \\
&\leq \max(c_1, ..., c_{\lceil \log\log h \rceil}) n \cdot 2 \cdot 2 \cdot 2^{\log\log h} - 2 \\
&= \max(c_1, ..., c_{\lceil \log\log h \rceil}) n \cdot 2 \cdot 2 \cdot \log h - 2 && a^{\log(b)} = b^{\log(a)} \\
&= O(n \log h)
\end{aligned}
$$

□

Corollary 4.1. CHANS-ALGORITHM under SQUARING-STRATEGY is time-optimal

Proof. Follows directly from the lower bound of the convex hull problem (theorem 3.4) and time complexity of CHANS-ALGORITHM under SQUARING-STRATEGY (theorem 4.7) □

Theorem 4.9 (Correctness of CHANS-ALGORITHM under SQUARING-STRATEGY)**.** CHANS-ALGORITHM under SQUARING-STRATEGY solves the convex hull sequence problem.

Proof. We know that SQUARING-STRATEGY will terminate at some point since $h \leq n$ is finite and the running time of CHANS-ALGORITHM is finite. SQUARING-STRATEGY does not terminate before $L \neq$ INCOMPLETE, which means that it suffices to show that CHANS-ALGORITHM returns CHV(Q) when $h' \geq h$.

We know that $p_1 \in \text{CHV}(Q)$, since it is has the lowest y-coordinate of all points in Q, or the leftmost such point in case of a tie. Consider any point $p \in Q$ that is not in $\bigcup_i \text{CHV}(Q_i)$. Then p is not in any CHV(Q_i) and since $Q_i \subseteq Q$, it cannot be an extremal point of CHV(Q). This is illustrated in figure 9. Thus, $\text{CHV}(Q) = \text{CHV}(\bigcup_i Q_i) = \text{CHV}(\bigcup_i S_i)$, which means that it suffices to find $\text{CHV}(\bigcup_i S_i)$. From theorem 4.6 we have that lines 8-10 constitute the same as a single call to MOST-CLOCKWISE-FROM$(p', \bigcup_i S_i)$. Correctness of JARVIS-MARCH concludes the proof.

□

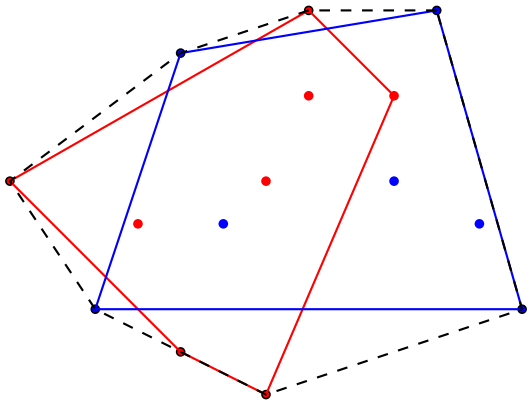


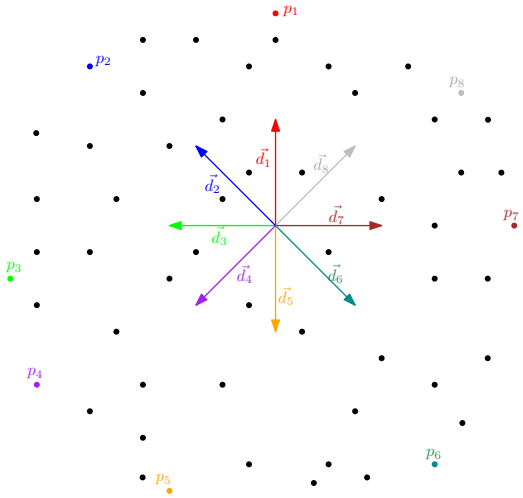
Figure 9: Example of Q partitioned into subsets Q_1 (red) and Q_2 (blue) with corresponding subhulls. The black dashed line is the convex hull of Q

5 Probabilistic Convex Hull

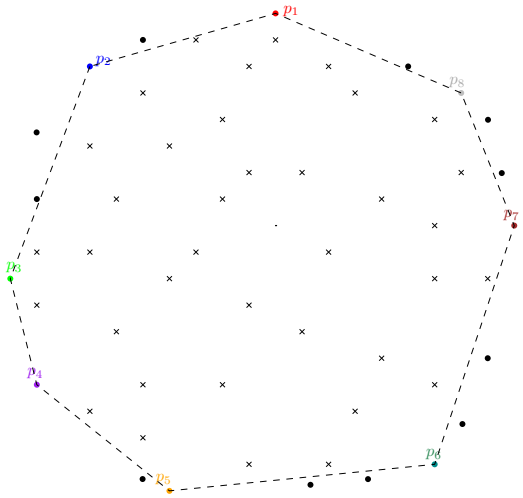
With deterministic algorithms for convex hull problems being lower-bounded by $\Omega(n \log h)$ we turn our efforts to a probabilistic version of convex hull. In this section we treat the points in Q as random variables that are independent identically distributed with some density. We denote these random variables $Z_1, Z_2, ..., Z_n$. Let $f(z)$ be the probability density function of one such Z_i landing in some coordinate $z = (x, y)$. This gives a probabilistic aspect to the problem and makes it possible to find an expected time complexity. Note that we already have linear expected time in JARVIS-MARCH and CHANS-ALGORITHM if f is a distribution such that $E[h] = O(1)$. Recall that time complexity of GRAHAM-SCAM is bottlenecked by the sorting step, so if f is a known distribution with structure that can be exploited to sort in linear time, this also achieves expected linear time. In this section we will show how convex hulls can be computed in expected linear time for various *rectangular* distributions of points by use of a simple heuristic first described by Akl and Toussaint [1]. We introduce the THROW-AWAY algorithm, a generalized version of the Akl and Toussaint heuristic.

5.1 The Throw-Away algorithm

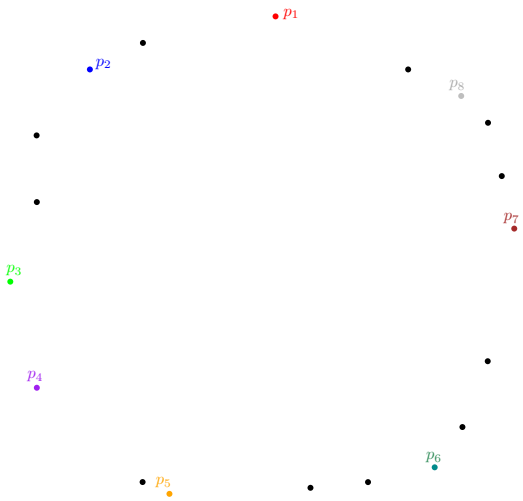
The simple idea of THROW-AWAY is to quickly find points in Q which can not be in CHV(Q) and eliminate ("throw away") these from further consideration. The algorithm takes two inputs $D = \{\vec{d_1}, \vec{d_2}, ..., \vec{d_k}\}$ and $Q = [q_1, q_2, ..., q_n]$. The set D contains directions represented by 2-dimensional vectors sorted in counter clockwise order. The algorithm finds the k points $[p_1, ..., p_k]$ in Q that lie furthest in each direction of D. Some of these may coincide which does not matter for correctness or asymptotic complexity, but in practical implementations it might be worthwhile to remove duplicates. The points form a convex polygon contained in the convex hull. Then all points of Q inside the polygon are eliminated, since none of these can be a member of CHV(Q), and the remaining points are returned. Figure 10 illustrates the steps of THROW-AWAY.



(a) Step 1: Find the extremal points $p_1, ..., p_k$ of the directions $\vec{d_1}, ..., \vec{d_k}$.



(b) Step 2: Eliminate all points inside the polygon formed by $p_1, ..., p_k$.



(c) Step 3: Return the remaining points.

Figure 10: The 3 steps of THROW-AWAY(D, Q) for 8 directions.

In the following, we let $\vec{q_i}$ be the vector representation of the point q_i and use the algorithm SEGMENTS-INTERSECT(e, l) from CLRS [4, p. 1018] which returns TRUE if the segments e and l intersect and FALSE if they do not.

Algorithm 5 THROW-AWAY(D, Q)

```
1:  for j = 1 to |D| do
2:      let p_j = q ∈ Q that maximizes d_j · q
3:  let P be the polygon formed by ⟨p_1, ..., p_k⟩
4:  let L be the largest horizontal distance between points in Q
5:  let Q' = Q
6:  for q in Q do
7:      intersections = 0
8:      let l be the horizontal line from q of length L
9:      for each edge e in P do
10:         if SEGMENTS-INTERSECT(e, l) then
11:             intersections += 1
12:     if intersections = 1 and q not a vertex of P then
13:         remove q from Q'
14: return Q'
```

Theorem 5.1 (Time complexity of THROW-AWAY)**.** THROW-AWAY has a worst case time complexity of $O(nk)$

Proof. The for-loop on lines 1-2 in THROW-AWAY iterates k times and finding p_j at each iteration takes n steps. Thus, line 1-3 takes c_1nk steps. Finding maximum horizontal distance on line 4 can be implemented in c_2n steps by a reduction that maintains maximum and minimum x-coordinate. Defining Q' on line 5 takes c_3 steps (or c_3n steps if a copy is desired). The for-loop on lines 6-13 iterates n times. Lines 7-8 takes c_4 steps. The nested for-loop on lines 9-11 iterates k times and the if-statement, where SEGMENTS-INTERSECT is called and intersections is incremented, takes c_5 steps. Thus, the nested for-loop takes c_5k steps. The if-statement on lines 12-13 takes c_6 steps. Thus, the for-loop on lines 6-13 takes $n(c_4 + c_5k + c_6)$ steps. Finally, the return statement takes c_7 steps.

Adding the running times analysed above, we get that THROW-AWAY has a running time of:

$$T(\text{THROW-AWAY}) = c_1nk + c_2n + c_3 + n(c_4 + c_5k + c_6) + c_7 = O(nk)$$

□

Correctness relies on the following simple lemma.

Lemma 5.1.1. The polygon $P = \langle p_1, ..., p_k \rangle$ found in lines 1-3 of THROW-AWAY is convex.

Proof. For each direction $\vec{d}$ we find a point p that maximizes $\vec{d} \cdot \vec{p}$. A fundamental property of the dot product is

$$\vec{d} \cdot \vec{p} = ||\vec{d}||||\vec{p}|| \cos(\theta)$$

where θ is the angle between the vectors. This is also the projection of $\vec{p}$ on $\vec{d}$, scaled by $||\vec{d}||$. Since $||\vec{d}||$ is constant for each direction, we are maximizing the projection which is exactly what we want in order to gain the point furthest in the direction of $\vec{d}$.

A simple proof by contradiction shows that the found points form a convex polygon. Assume for contradictory purposes that the formed polygon is not convex, then there must be at least one p which is a concave corner but has a convex corner neighbor. But maximizing in any direction, even parallel with $\vec{p}$, would then never give this point, but rather the neighbor. If more concave corners exist, the same argument works in an inductive manner.

Conclusively, we have that $P = \langle p_1, ..., p_k \rangle$ forms a convex polygon. □

Theorem 5.2 (Correctness of THROW-AWAY)**.** The THROW-AWAY procedure does not remove any vertex of the convex hull. That is, CHV(THROW-AWAY(Q)) = CHV(Q).

Proof. By lemma 5.1.1, polygon $P = \langle p_1, \dots p_k \rangle$ is convex and thus represents a convex set of points. By definition of the convex hull, the convex set represented by P is a subset of the convex hull. Further, all points inside P, that are not a vertex of P, cannot be a vertex in the convex hull, because this would make the hull non-convex. Since the algorithm only removes points on line 13, it suffices to show that the if-statement on line 12 is only true if q is in the polygon P but not a vertex. Clearly, the secondary condition is checked. It remains to show that intersection being set to 1 implies that q is inside P. From figure 10(b) it is rather obvious that this is the case. There are some edge cases that need to be handled by SEGMENTS-INTERSECT, for instance when a point is on one of the edges of P, but these cases can be solved in constant time.

Conclusively, we have that THROW-AWAY eliminates all non-vertex points inside the polygon $P = \langle p_1, ..., p_k \rangle$ and returns the remaining points which contain CHV(Q). □

5.2 Computing the convex hull in expected linear time

The principle behind THROW-AWAY can be traced back to Selim Akl and G. T. Toussaint in 1978. The so-called *Akl-Toussaint heuristic* is a special case of THROW-AWAY where the 8 extremal points of Q are found by by maximizing along the axis aswell as diagonal between axis. This is equivalent to using

$$D = \{(1,0), (-1,0), (0,1), (0,-1), (1,1), (-1,-1), (1,-1), (-1,1)\}$$

as input to THROW-AWAY. Since $k = 8$, we have that the running time is $O(n)$. This can be used as a preprocessing step in practical applications [] but can also be used to prove expected linear time for certain distributions.

Let C be the time complexity of running a convex hull algorithm on some data after it has been preprocessed by the Akl-Toussaint heuristic and let $E[C]$ be the expected complexity. In this section we will prove linear expected time if f is a distribution inside a rectangle. The proof is strongly inspired by the proof derived by L. Devroye and G. T. Toussaint [5].

Lemma 5.2.1. Let $0 < m \leq f(z) \leq M < \infty$ for all z in some non-degenerate rectangle of $\mathbb{R}^2$ and let $f(z) = 0$ elsewhere. Let N be the number of points returned after after applying the Akl-Toussaint heuristic on Q. Then for all positive ϵ we have,

$$P(N > \epsilon) \leq k_1 \exp(-\frac{k_2 \epsilon^2}{n})$$

for positive constants k_1 and k_2

Proof. It suffices to show the proof for the rectangle with corner in $(0,0)$ and $(1,1)$ as anything else is a matter of scaling and translating. Let $(X_1,Y_1),...,(X_4,Y_4)$ be the extremal points along diagonal of axis (towards the corners of the rectangle as in figure 11). Draw 4 horizontal and 4 vertical lines through the four extremal points and let T be the rectangle inside the 4 innermost lines. Let N_T be the number of Z_i's outside T. Let $T_1,...,T_8$ be the rectangular strips between the 8 lines and the nearest edge of the unit square. Let $N_1,...,N_8$ be the number of Z_i's in $T_1,...,T_8$ and let $A_1,...,A_8$ be the probabilities of a point landing in the respective area of $T_1,...,T_8$.

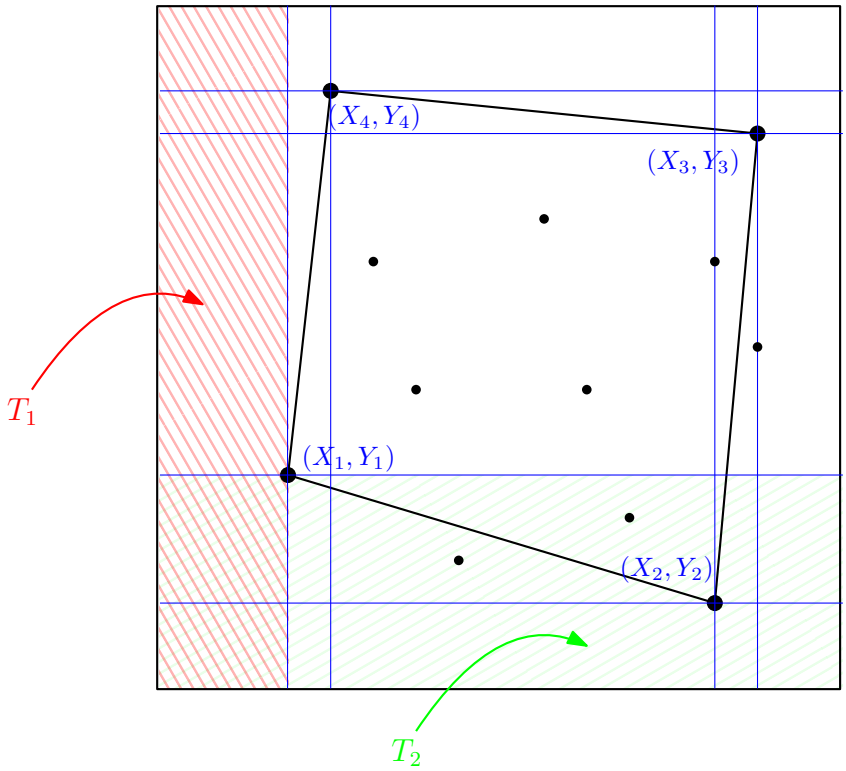


Figure 11: Shaded areas illustrate 2 stripes of $T_1,...,T_8$

We have that $N \leq N_T$ because the area outside the polygon defined by $(X_1,Y_1),...,(X_4,Y_4)$ is a subset of the area outside T. Further, we have $N_T \leq \sum_{i=1}^{8} N_i$ because the area of outside T is the same area as $\bigcup_{i=1}^{8} T_i$, but the T_i's overlap at some points, which means that some points may be overcounted in $\sum_{i=1}^{8} N_i$. This allows the following bound,

$$
\begin{aligned}
& P(N > \epsilon) \\
\leq\, & P(N_T > \epsilon) && N \leq N_T \\
=\, & P(\frac{N_T}{8} > \frac{\epsilon}{8}) \\
\leq\, & \sum_{i=1}^{8} P(N_i > \frac{\epsilon}{8}) && \max(N_i) \geq \frac{N_T}{8} \\
=\, & \sum_{i=1}^{8} P(N_i - nA_i + nA_i > \frac{\epsilon}{16} + \frac{\epsilon}{16}) \\
=\, & \sum_{i=1}^{8} P(N_i - nA_i > \frac{\epsilon}{16} \vee nA_i > \frac{\epsilon}{16}) \\
\leq\, & \sum_{i=1}^{8} P(N_i - nA_i > \frac{\epsilon}{16}) + P(nA_i > \frac{\epsilon}{16}) && \text{Union bound}
\end{aligned}
$$

Let T_1 be the rectangular strip defined by the corners $(0, 0)$ and $(Y_1, 1)$ (red area on figure 11). Let F be the distribution function of the x-component of Z_1, that is:

$$F(z) = \int_0^x \int_0^1 f(x, y)\, dy\, dx$$

Let F_n be the empirical distribution function of the x-components, that is:

$$F_n(z) = \frac{1}{n} \times |\{Z_i : X_i \leq x\}|$$

We have that $F_n(z)$ is the empirical distribution of $F(z)$, which allows the following two bounds,

$$
\begin{aligned}
& P(N_1 - nA_1 > \frac{\epsilon}{16}) \\
=\, & P(\frac{N_1}{n} - A_1 > \frac{\epsilon}{16n}) \\
=\, & P(F_n(Z_1) - F(Z_1) > \frac{\epsilon}{16n}) \\
\leq\, & P(\sup |F_n(Z_1) - F(Z_1)| > \frac{\epsilon}{16n}) \\
\leq\, & k_3 \exp(-2n(\frac{\epsilon}{16n})^2) && \text{Dvoretzky–Kiefer–Wolfowitz inequality} \\
=\, & k_3 \exp(-\frac{\epsilon^2}{128n})
\end{aligned}
$$

as well as,

$$
\begin{aligned}
& P(nA_1 > \frac{\epsilon}{16}) \\
\leq & P(A_1 > \frac{\epsilon}{16n}) \\
\leq & P(MX_1 > \frac{\epsilon}{16n}) && \text{Since } X_1 \cdot 1 \text{ is the size of } T_1 \\
= & P(X_1 > \frac{\epsilon}{16nM}) \\
\leq & P(X_1 + Y_1 > \frac{\epsilon}{16nM}) \\
= & P(\text{triangle } (0,0), (0, \frac{\epsilon}{16nM}), \\
& (\frac{\epsilon}{16nM}, 0) \text{ is empty}) && \text{Triangle trick: explained below} \\
\leq & (1 - \frac{1}{2}(\frac{\epsilon}{16nM})^2 m)^n && \text{Probability calculation: explained below} \\
\leq & (\exp(-\frac{1}{2}(\frac{\epsilon}{16nM})^2 m))^n && x \leq e^x - 1 \\
= & \exp(-(\frac{\epsilon^2 m}{512nM^2}))
\end{aligned}
$$

Triangle trick: $X_1 + Y_1$ forms a triangle $(0,0)$, $(0,Y_1)$, $(X_1,0)$ where (X_1,Y_1) is on the edge. Since (X_1,Y_1) is maximal along the diagonal, no other point can be in this triangle and if just one point is in $(0,0)$, $(0,\frac{\epsilon}{16nM})$, $(\frac{\epsilon}{16nM},0)$, then (X_1,Y_1) is in the triangle and thus $X_1 + Y_1 \leq \frac{\epsilon}{16nM}$. This means that $[X_1+Y_1 > \frac{\epsilon}{16nM}] \Leftrightarrow [\text{triangle } (0,0), (0,\frac{\epsilon}{16nM}), (\frac{\epsilon}{16nM},0) \text{ is empty}]$.

Probability calculation: The probability of one point being in the triangle $(0,0), (0,\frac{\epsilon}{16nM}), (\frac{\epsilon}{16nM},0)$ is the size of the rectangle minus size of triangle multiplied with the average probability density of the triangle. This is upper-bounded by $1 - \frac{1}{2}(\frac{\epsilon}{16nM})^2 m$ since for all z in the rectangle $m \leq f(z)$. This probability is multiplied with itself n times, since $Z_1, ..., Z_n$ are i.i.d.

Note that the two bounds hold independently of strips $T_1, ..., T_8$. Conclusively, we get that,

$$
\begin{aligned}
& P(N > \epsilon) \\
\leq & \sum_{i=1}^{8} P(N_i - nA_i > \frac{\epsilon}{16}) + P(nA_i > \frac{\epsilon}{16}) \\
\leq & \sum_{i=1}^{8} k_3 \exp(-\frac{\epsilon^2}{128n}) + \exp(-(\frac{\epsilon^2 m}{512nM^2})) \\
= & 8(k_3 \exp(-\frac{\epsilon^2}{128n}) + \exp(-(\frac{\epsilon^2 m}{512nM^2}))) \\
= & k_1 \exp(-\frac{k_2 \epsilon^2}{n}) && M, m \text{ are constants}
\end{aligned}
$$

□

Theorem 5.3 (Expected running time using the Akl-Toussaint heuristic)**.** Preprocessing Q with the Akl-Toussaint heuristic and then using a $O(n^2)$ convex hull algorithm has $E[C] = O(n)$ for any densities bounded by $0 < m \leq f(z) \leq M < \infty$ for all z in some non-degenerate rectangle of $\mathbb{R}^2$ and let $f(z) = 0$ elsewhere, where m and M are constants.

Proof. Firstly, it is clear that $C \leq O(n) + O(N^2)$, since the Akl-Toussaint heuristic has a time complexity of $O(n)$ and applying an $O(n^2)$ convex hull algorithm on the remaining N

points has a time complexity of $O(N^2)$. Thus, we want to show that $E[N^2] = O(n)$. Since N is a discrete random variable, we have that:

$$
\begin{aligned}
& E[N^2] \\
&= \sum_{t=0}^{\infty} P(N^2 = t)t \\
&= \sum_{u=0}^{\infty} P(N^2 = u^2)u^2 && \text{Square trick: explained below} \\
&= \sum_{u=0}^{\infty} P(N = u)u^2 && N \geq 0 \\
&= \sum_{u=0}^{\infty} (u^2 - u)P(N = u) + \sum_{u=0}^{\infty} uP(N = u) \\
&= \sum_{u=0}^{\infty} 2\frac{u^2 - u}{2} P(N = u) + \sum_{u=0}^{\infty} uP(N = u) \\
&= \sum_{u=0}^{\infty} 2u \sum_{u'=u+1}^{\infty} P(N = u') + \sum_{u=0}^{\infty} uP(N = u) \\
&= \sum_{u=0}^{\infty} 2uP(N > u) + \sum_{u=0}^{\infty} uP(N = u) \\
&= \sum_{u=0}^{\infty} 2uP(N > u) + \sum_{u=0}^{\infty} P(N > u) \\
&= \sum_{u=0}^{\infty} (2uP(N > u) + P(N > u)) \\
&= \sum_{u=0}^{\infty} (2u + 1)P(N > u)
\end{aligned}
$$

Square trick: Firstly, we have that $t = u^2$. We have that t is always an integer and therefore u^2 is also always an integer. The sum goes from 0 to ∞, which means that all the occurrences of t will eventually occur when summing over $u = \sqrt{t}$.

This intermediate result differs very slightly (in an inconsequential way) from what Devroye and Toussaint derive in their proof. We believe this is a small error in Devroye and Toussaints paper, which luckily is remedied by asymptotics.

Using lemma 5.2.1 and the above derivation,

$$
\begin{aligned}
& E[N^2] \\
=& \sum_{u=0}^{\infty}(2u+1)P(N>u) \\
\leq& \sum_{u=0}^{\infty}(2u+1)k_1\exp(-\frac{k_2u^2}{n}) \\
=& \int_0^{\infty}(2u+1)k_1\exp(-\frac{k_2u^2}{n})du \\
=& 2k_1\int_0^{\infty}u\exp(-\frac{k_2u^2}{n})du + \int_0^{\infty}u\exp(-\frac{k_2u^2}{n})du \\
=& \frac{k_1}{k_2}n - \frac{1}{2k_2}n \\
=& O(n)
\end{aligned}
$$

Conclusively, we have that $E[N^2] = O(n)$. Thus, $E[C] \leq O(n) + E[N^2] = O(n)$.

□

6 Experiments

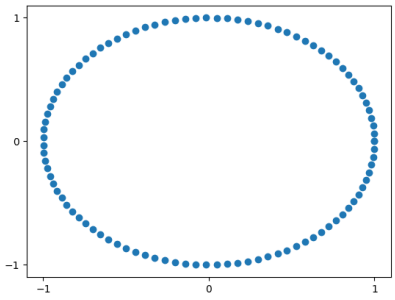
So far we have introduced various algorithms that solve the convex hull sequence problem and proven their correctness and asymptotic time complexity. This section takes a more practical approach, exploring whether theory matches reality. We test the algorithms described in section 4 as well as Chans-Algorithm using Throw-Away with 8 directions as preprocessing. Each of the algorithms are compared on a selection of input distributions for increasing values of n. Instead of measuring runtime we measure the number of orientation tests performed. Specifically, each time the cross product is calculated in $p_1 \circlearrowleft_p p_2$ a counter is incremented. Note that Segments-Intersect of Throw-Away performs 4 orientation tests. We argue that the number of orientation tests is a good complexity estimator for convex hull algorithms, since it is used in the dominating term of the cost analysis of all the algorithms. This approach eliminates the need to analyse concepts such as *locality*, *memory bottlenecks*, *branch prediction* and *clock speed*. This is not to say that these aspects are not important, but it is removed from the focus of the thesis and unrealistic for us to cover in any fulfilling capacity given the scope of the thesis.

The algorithms are run on the following four distributions (also illustrated in figure 12):

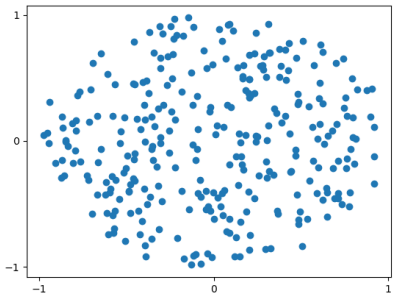
- **Circle**: Evenly distributed on circle: $Z = (X, Y) \in \{Z : X^2 + Y^2 = 1\}$
- **Disk**: Uniform sampling from disk $Z = (X, Y) \in \{Z : X^2 + Y^2 \leq 1\}$
- **Square**: Uniform sampling from square $Z = (X, Y) \in \{Z : 0 \leq X \leq 1 \wedge 0 \leq Y \leq 1\}$
- **Triangle**: Uniform sampling from square inscribed in a triangle

These distributions are selected since they cover a variety of sizes for the convex hull. In one extreme, the circle distribution will have $h = n$ which is the worst case for Jarvis-March and Chans-Algorithm. On the other extreme the triangle distribution will have $h = 3$, which is the best case for these algorithms. The disk and square distributions are between these extremes with $E[h] = O(\sqrt[3]{n})$ for disk and $E[h] = O(\log n)$ for square [8]. Graham-Scan is independent of h, but it should perform relatively better on large h, although it should always be worse than Chans-Algorithm since $h \leq n$. The uniform distribution in unit square is also interesting since by theorem 5.3 it should result in linear time for Throw-Away combined with any of the algorithms.

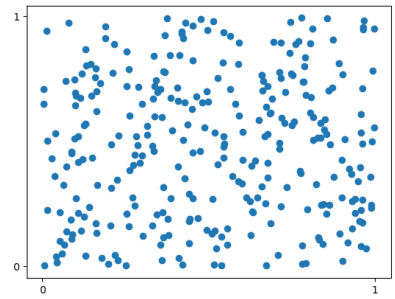
Correctness was tested by running each of the algorithms on the distributions and visually determining if anything looked off. This is illustrated in figure 13, where each of the distributions in figure 12 is solved by one of the four algorithms we are experimenting with. This is obviously not a foulproof way of ensuring correctness, so it should be understood as a possible source of error. The workhorse experiments were done on different input sizes from 10^3 increasing in powers of ten up to an amount where runtime exceeded what our available hardware can handle in a reasonable time frame, which was usually around 10^7 data points. Results are plotted in a log-log plot since this allows all the algorithms to be contained in the same plot. Note that on a log-log plot, linear growth looks like a line of slope 1, while quadratic growth also look linear but with slope 2. We have drawn lines between datapoints, but for Chans-Algorithm there is likely a sawtooth slope between the points, due to the Squaring-Strategy. It should be noted that there is some variance from the sampling, which is not evident from the plots. Raw data from experiments are in appendix 8.1 and the code in `bsc_thesis_code.zip`.



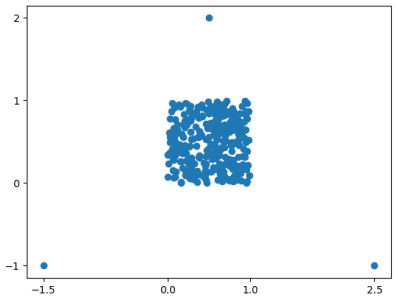
(a) Circle distribution with $n = 100$



(b) Disk distribution with $n = 300$

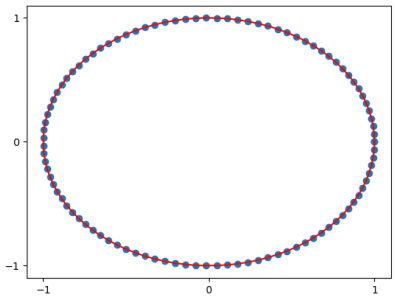


(c) Square distribution with $n = 300$

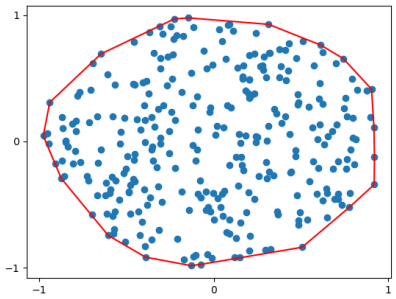


(d) Triangle distribution with $n = 300$

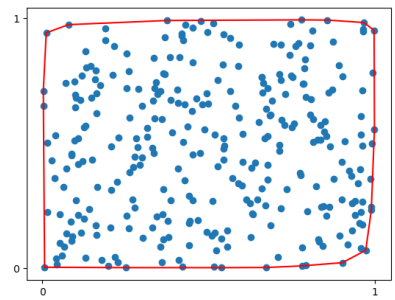
Figure 12: Showcase of distributions used in experiments



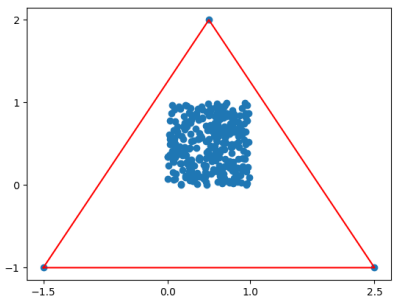
(a) Convex hull of circle distribution



(b) Convex hull of disk distribution



(c) Convex hull of square distribution



(d) Convex hull of triangle distribution

Figure 13: Showcase of algorithms used in experiments

6.1 Results

Looking at figure 14, we see that the performances vary a lot. The complexity of Jarvis-March grows quadratically. This is expected since $h = n$ makes Jarvis-March runtime $O(nh) = O(n^2)$. In accordance with expectations, the other three algorithms have same growth (are parallel) but are all faster (smaller slope) than Jarvis-March. The gap between Chans-Algorithm and Graham-Scan is likely due to constant factors.

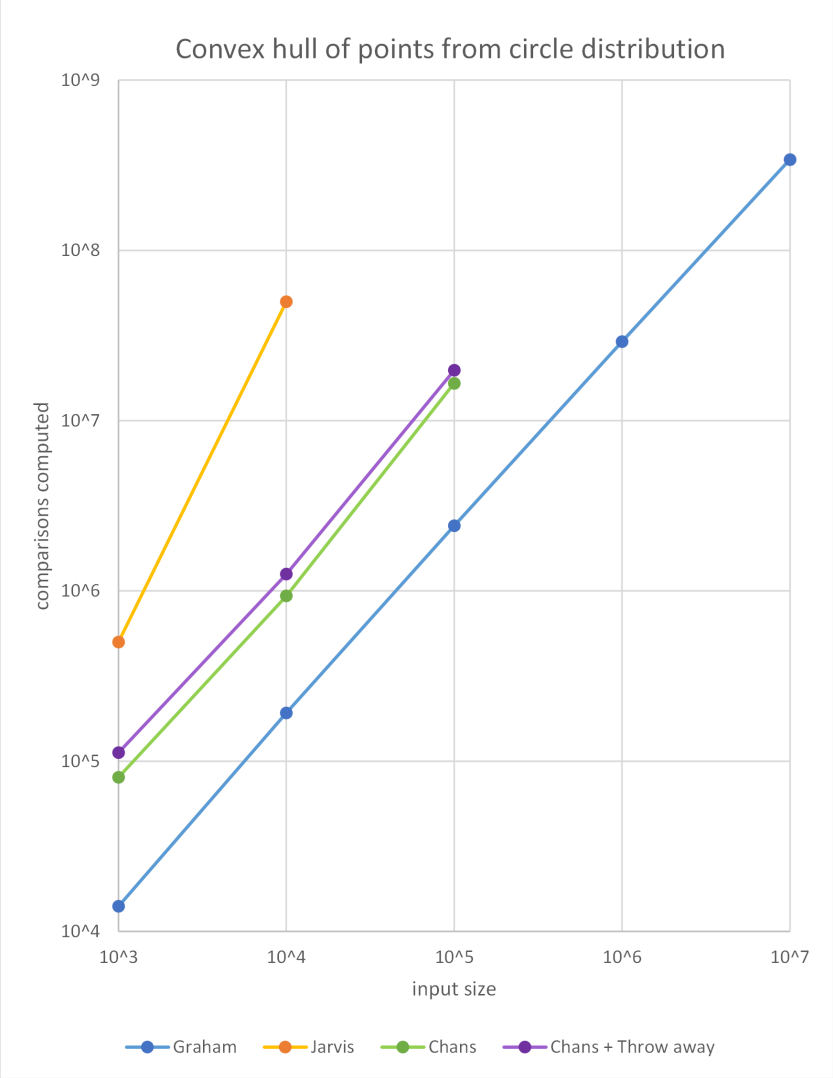


Figure 14: Performance of convex hull algorithms on circle distribution

Observing figure 15, the disk distribution performances are much closer. We see that Graham-Scan works well for small input sizes, but is eventually overtaken by Chans-Algorithm with Throw-Away. This is surprising, since with $E[h] = O(\sqrt[3]{n})$, the expected complexity for Chans-Algorithm is $O(n \log(\sqrt[3]{n})) = O(\frac{n \log(n)}{3}) = O(n \log(n))$, which is equal to the complexity of Graham-Scan. Chans-Algorithm follows well for small input sizes, but eventually jumps at $n = 10^6$, which is surprising. Jarvis-March performs much better than it did with input from the circle distribution, in fact the expected complexity is $O(n\sqrt[3]{n}) = O(n^{\frac{4}{3}})$.

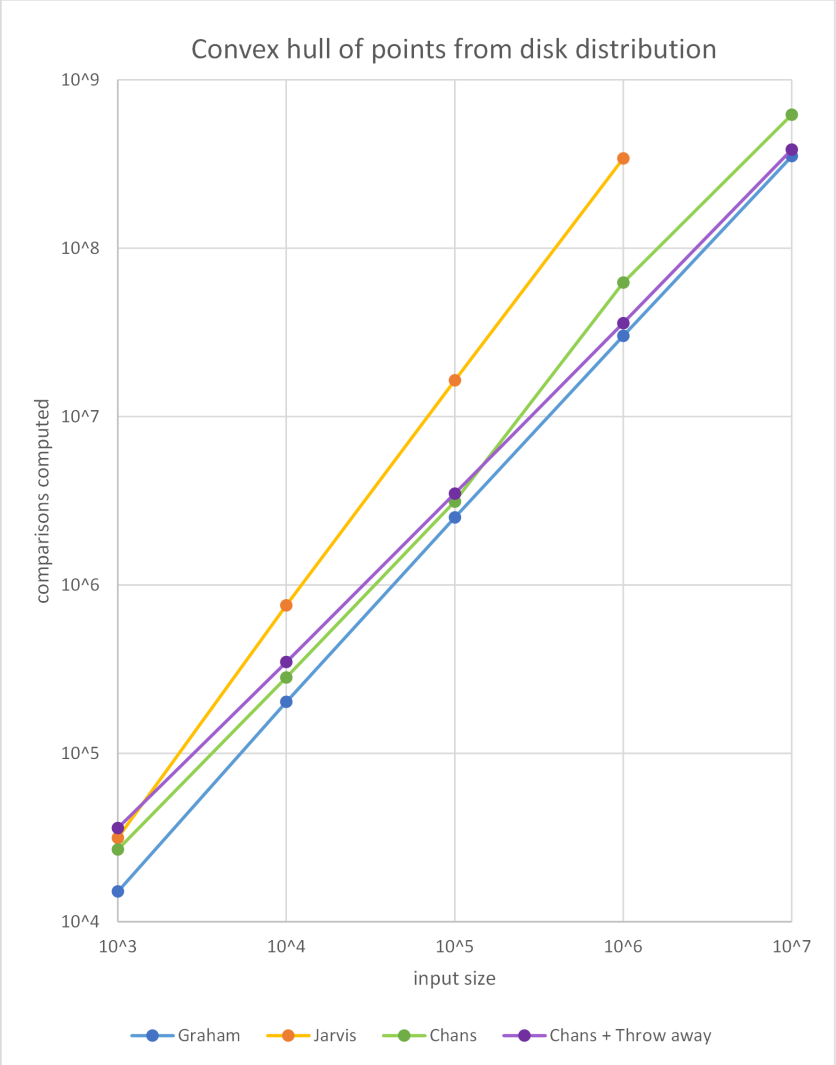


Figure 15: Performance of convex hull algorithms on disk distribution

Figure 16 shows our results for the square distribution. We observe that the algorithms have a very similar number of comparisons for each value of n, but there is still a clear tendency for small and large values of n; For small values of n, Jarvis-March and Graham-Scan performs well, but as n increases, Chans-Algorithm and Chans-Algorithm + Throw-Away gets better results. In the interval $n \in [10^4, 10^7]$ we see that the curves of Jarvis-March and Graham-Scan are parallel and very close. This makes good sense, since $E[h] = O(\log(n))$, so they have same asymptotic running times. Looking at Chans-Algorithm and Chans-Algorithm with Throw-Away, they are parallel and it is not possible to see whether the graph of Chans-Algorithm + Throw-Away will overtake the graph of Chans-Algorithm at some point. Since $E[h] = O(\log(n))$, we have that the expected running time of Chans-Algorithm is $O(n \log \log n)$, which is very close to the expected running time $O(n)$ of Chans-Algorithm + Throw-Away.

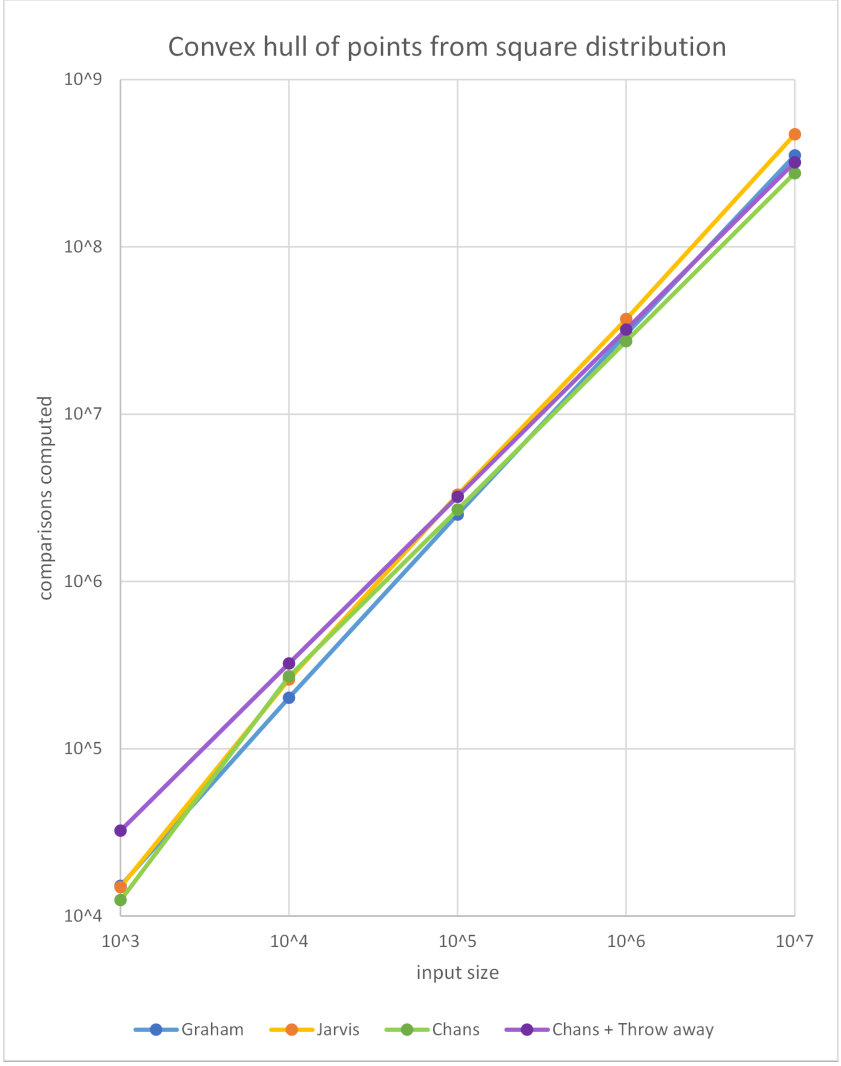


Figure 16: Performance of convex hull algorithms on square distribution

The experiments made on the triangle distribution, shown in figure 17, clearly shows how small values of h impacts the complexities of the algorithms. Whenever h is constant (in this case 3), Jarvis-March, Chans-Algorithm and Chans-Algorithm with Throw-Away all have the complexity $O(n)$. Graham-Scan is not affected by the reduction of h, which clearly shows in the plot.

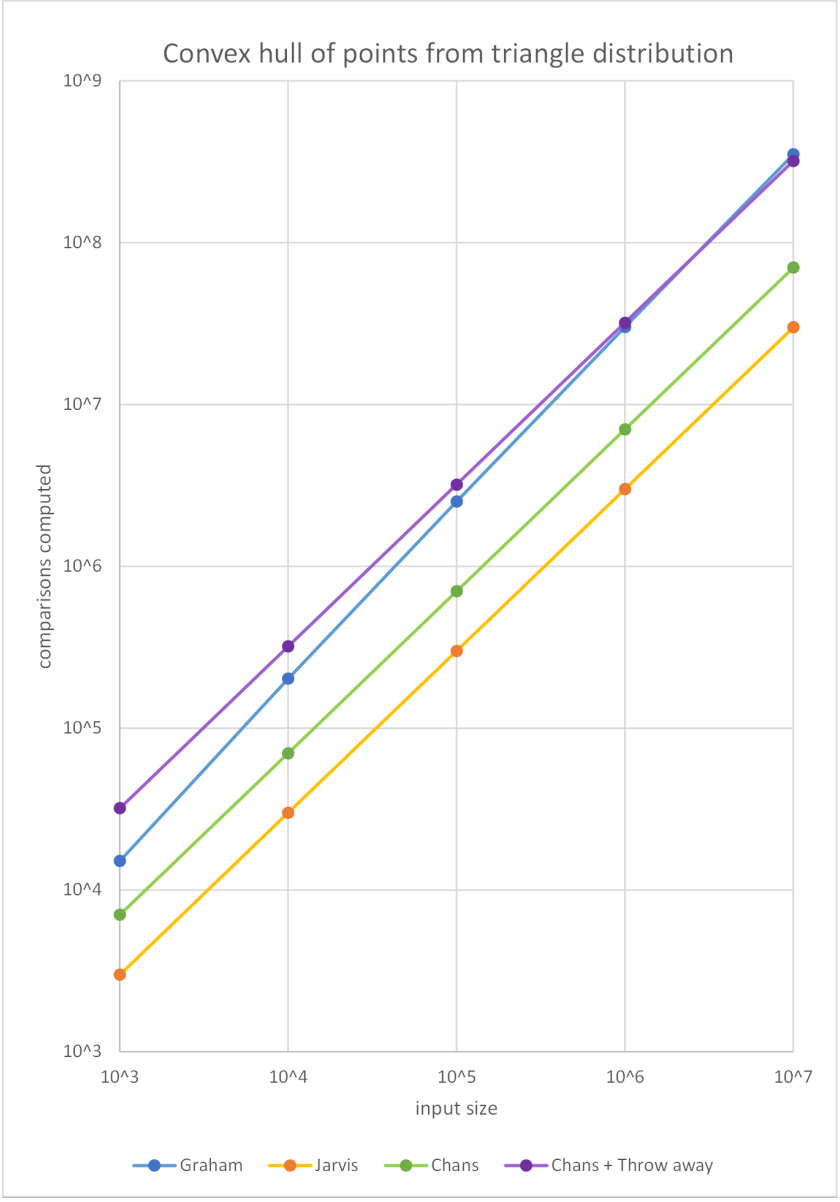


Figure 17: Performance of convex hull algorithms on triangle distribution

Overall, the experiments have given good insight on the performance of the algorithms with different values h. Even though we, by corollary 4.1, derived that Chans-Algorithm is time optimal, we now see that there are cases where other algorithms performs better in practice. In fact, the experiments showed that every algorithm we experimented with had a distribution it performed better on than the other algorithms. Perhaps the most surprising result is the strong performance of Chans-Algorithm even for small input sizes, which is not obvious when considering the amount of work that goes lost with Squaring-Strategy. The constants for Chans-Algorithm are not as big as we expected.

7 Concluding Remarks

We set out to cover the theoretical foundations of the planar convex hull problem in a cohesive manner. By proving the lower bound on a variety of algorithmic variants, describing time-optimal algorithms that solve the problem and analyzing a probabilistic variant, we believe this work is aptly concluded.

Further work in the spirit of this pursuit could analyze memory usage, convex hulls in higher dimensions, or dive deeper into distributions for which the convex hull can be computed quickly. In the forming of this thesis, we attempted to prove $o(n \log h)$ expected time for a general notion of radial distributions by using Throw-Away with $\log \log n$ directions in a similar manner to the proof for rectangular distributions. Ultimately we did not have success in this matter, but there is potential for future work.

The experiments in this thesis were a secondary concern and would benefit greatly from more time and effort, both methodically by more thorough correctness testing, and in scope by analyzing practical factors in greater detail. In a similar vein, we believe there is potential in a so-called *introspective* algorithm that integrates different algorithms to achieve practical running time speedups. With modern computers relying increasingly on parallelism, this is also an avenue for investigation. Lastly, our work shows that knowing the underlying input distribution can be used to speed up running times, so it would be very beneficial to investigate which distributions commonly appear in applications.

References

[1] Selim G. Akl and Godfried T. Toussaint. A fast convex hull algorithm. *Inf. Process. Lett.*, 7(5):219–222, 1978.

[2] Michael Ben-Or. Lower bounds for algebraic computation trees (preliminary report). In David S. Johnson, Ronald Fagin, Michael L. Fredman, David Harel, Richard M. Karp, Nancy A. Lynch, Christos H. Papadimitriou, Ronald L. Rivest, Walter L. Ruzzo, and Joel I. Seiferas, editors, *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, 25-27 April, 1983, Boston, Massachusetts, USA*, pages 80–86. ACM, 1983.

[3] Timothy M. Chan. Optimal output-sensitive convex hull algorithms in two and three dimensions. *Discret. Comput. Geom.*, 16(4):361–368, 1996.

[4] Mark de Berg, Otfried Cheong, Marc J. van Kreveld, and Mark H. Overmars. *Computational geometry: algorithms and applications, 3rd Edition*. Springer, 2008.

[5] Luc Devroye and Godfried T. Toussaint. A note on linear expected time algorithms for finding convex hulls. *Computing*, 26(4):361–366, 1981.

[6] Ask Neve Gamby and Jyrki Katajainen. Convex-hull algorithms: Implementation, testing, and experimentation. *Algorithms*, 11(12):195, 2018.

[7] Ronald L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Inf. Process. Lett.*, 1(4):132–133, 1972.

[8] Sariel Har-Peled. On the expected complexity of random convex hulls. *CoRR*, abs/1111.5340, 2011.

[9] Ray A. Jarvis. On the identification of the convex hull of a finite set of points in the plane. *Inf. Process. Lett.*, 2(1):18–21, 1973.

[10] M. A. Jayaram and H. Fleyeh. Convex hulls in image processing: A scoping review. *American Journal of Intelligent Systems*, 6(2):48–58, 2016.

[11] David G. Kirkpatrick and Raimund Seidel. The ultimate planar convex hull algorithm? *SIAM Journal on Computing*, 15(1):295–298, 1986.

[12] Rong Liu, Hao Zhang, and James Busby. Convex hull covering of polygonal scenes for accurate collision detection in games. In Chris Shaw and Lyn Bartram, editors, *Proceedings of the Graphics Interface 2008 Conference, May 28-30, 2008, Windsor, Ontario, Canada*, ACM International Conference Proceeding Series, pages 203–210. ACM Press, 2008.

[13] Nafiseh Masoudi, Georges M. Fadel, and Margaret M. Wiecek. Planning the shortest path in cluttered environments: A review and a planar convex hull-based approach. *J. Comput. Inf. Sci. Eng.*, 19(4), 2019.

8 Appendix

8.1 Experimental results

Input size	1000	10000	100000	1000000	10000000
Graham	14066	192252	2412089	29117524	341597556
Jarvis	500499	50004999			
Chans	80370	935654	16561033		
Chans + Throw away	112370	1255654	19761033		

Table 1: Experimental results from circle distribution

Input size	1000	10000	100000	1000000	10000000
Graham	15109	202048	2513071	30116605	351603432
Jarvis	31503	757149	16386633	340942029	
Chans	26844	282143	3127506	62443270	620456850
Chans + Throw away	36005	348179	3485185	35898945	384670362

Table 2: Experimental results from disk distribution

Input size	1000	10000	100000	1000000	10000000
Graham	15158	201950	2513068	30118476	351606275
Jarvis	14894	259674	3299471	36999333	469998918
Chans	12455	270513	2680766	27295728	275669676
Chans + Throw away	32448	324093	3207444	32037159	320105433

Table 3: Experimental results from square distribution

Input size	1000	10000	100000	1000000	10000000
Graham	15106	202195	2513134	30118194	351610734
Jarvis	2996	29996	299996	2999996	29999996
Chans	7025	69915	700977	7019537	70180652
Chans + Throw away	32022	320021	3200022	32000022	320000022

Table 4: Experimental results from triangle distribution